

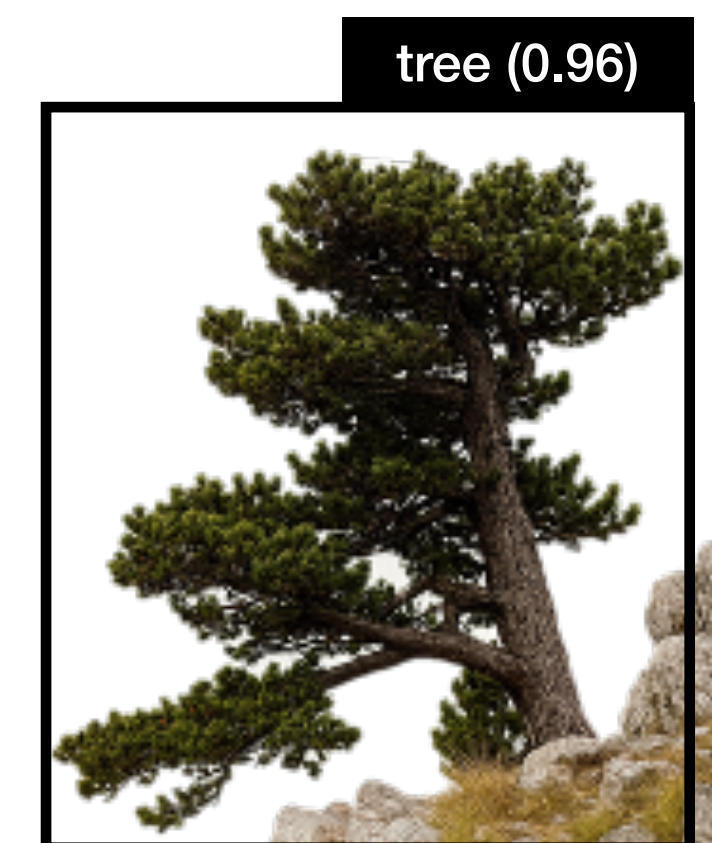
Foundations of Deep Learning

Vito Dichio, Fellow in AI (ENS-PSL)

✉ / vito.dichio@psl.eu

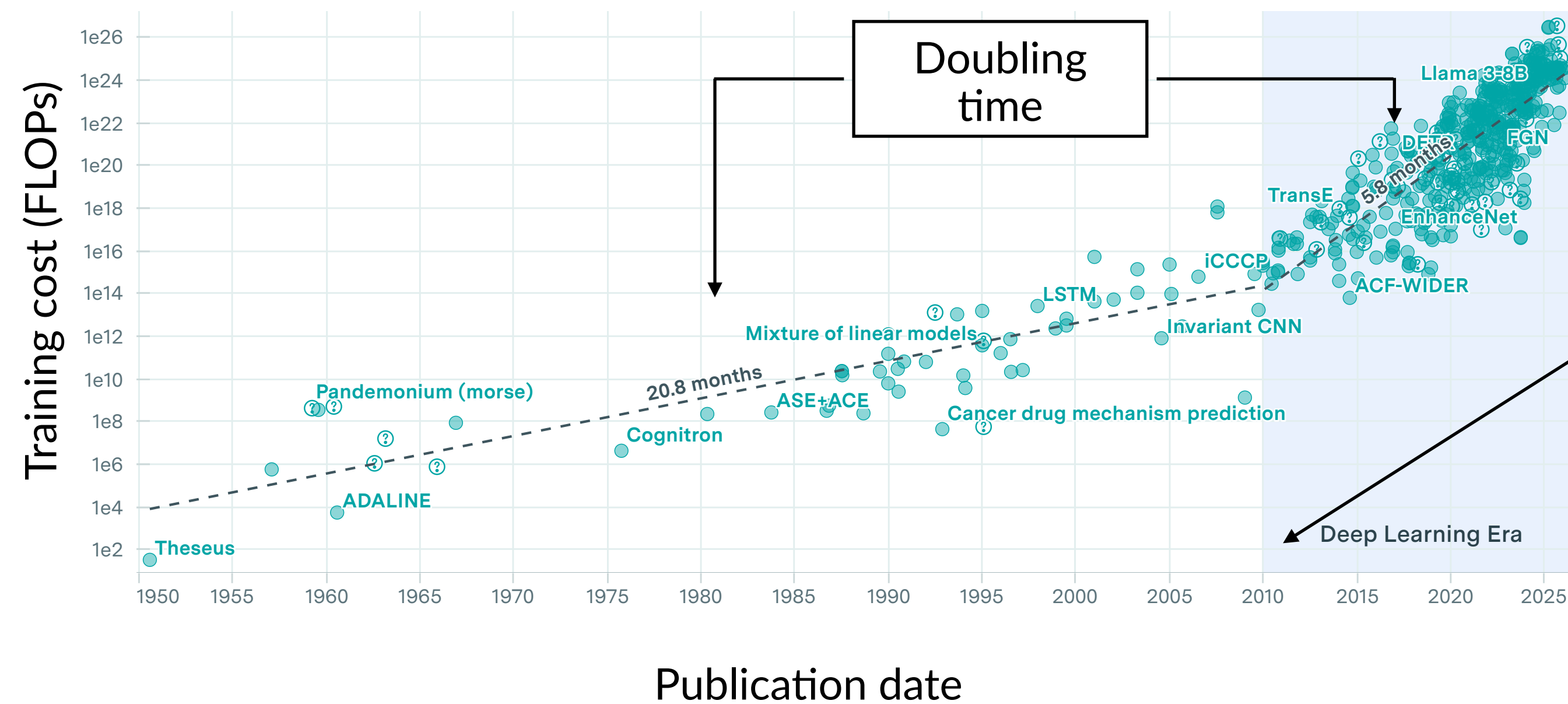
Lecture 3 : Losses

Master's Degree in **AI & Society** (PSL-PSAI, Spring 2026)



Last week

Two Eras of Deep Learning



Key ingredients:

- Algorithmic innovations
Residual connections, Dropout...
- Large-scale data
Ex.: ImageNET
- Hardware revolution
GPUs, TPUs ...

Limitations of Basis Functions

A large-enough set of $\{\phi_j(\mathbf{x})\}$ can approximate any function

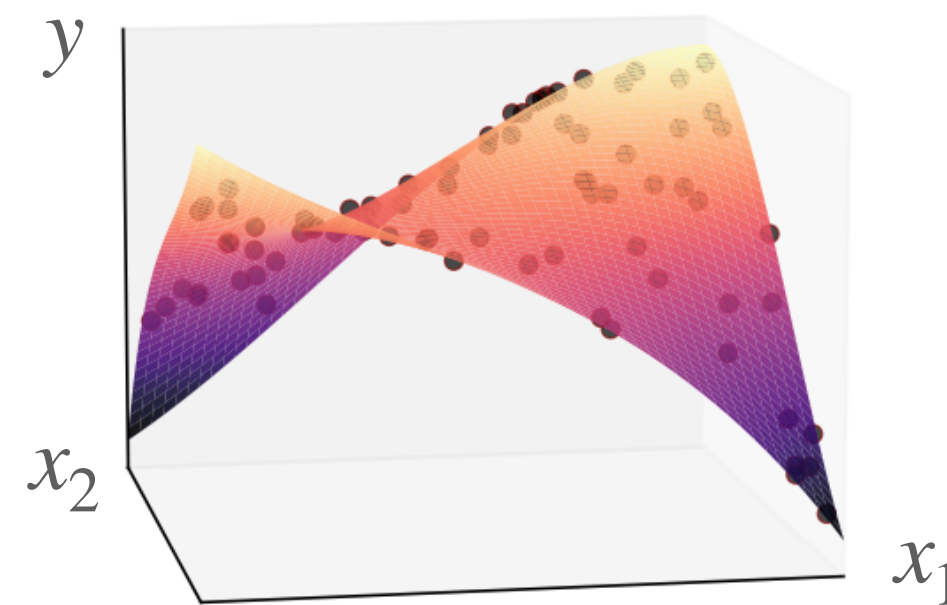
BUT

In high dimensions, **combinatorial explosion** of number of basis functions

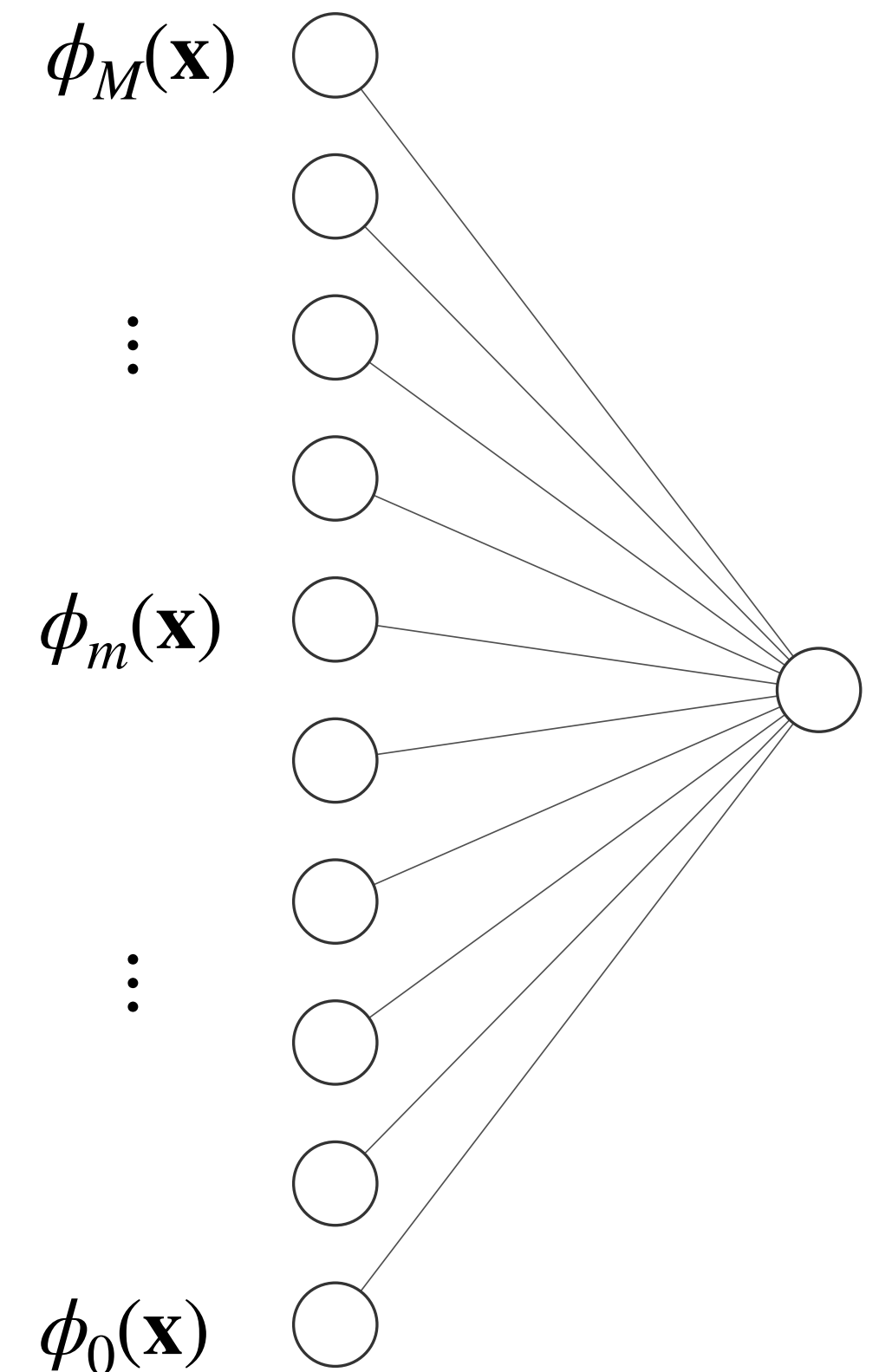
Fixed features may **not capture the true structure** of the data



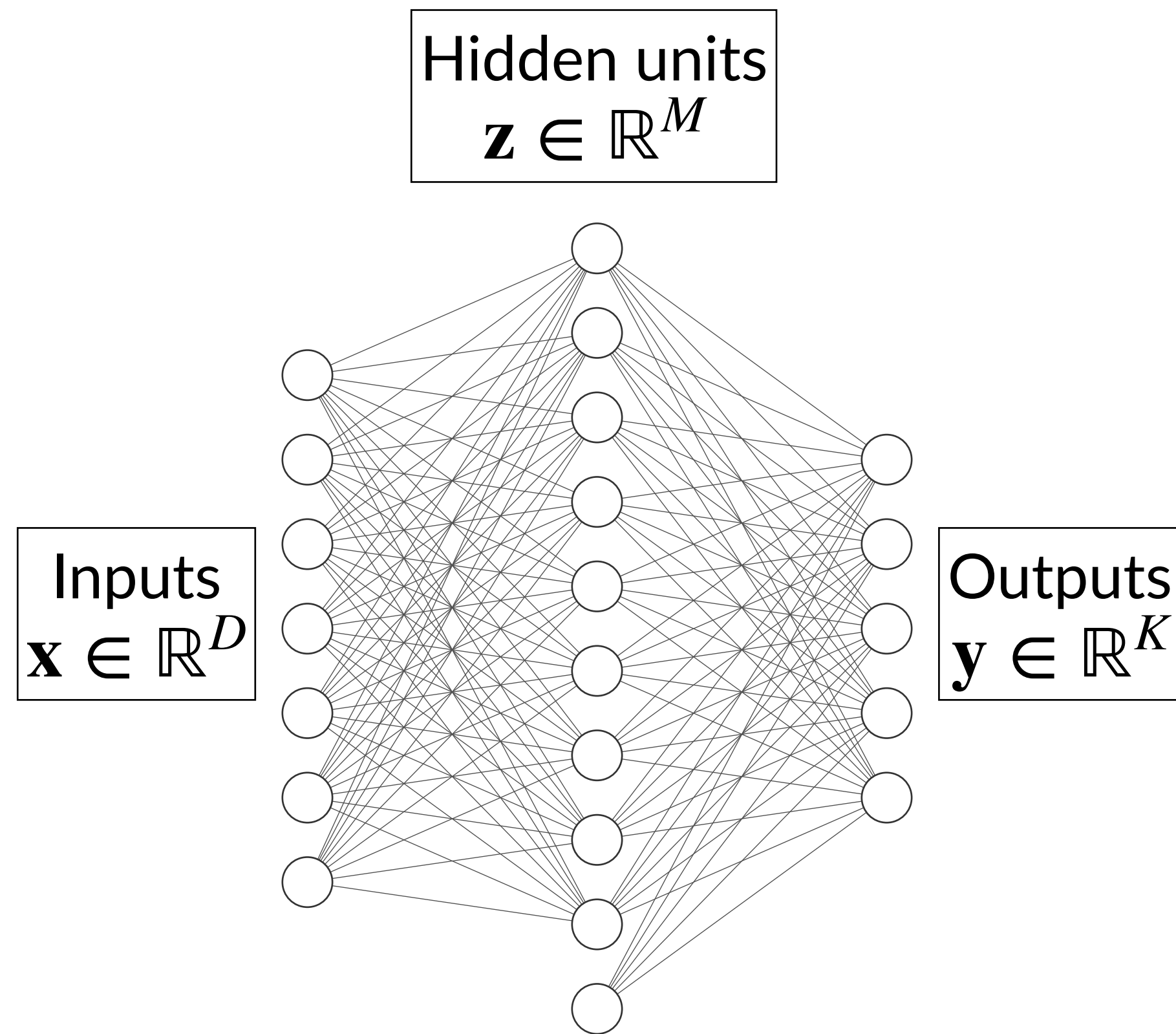
No need for pixel-level description,
three effective degrees of freedom



10 basis functions needed to fit a 2D
curve with a polynomial of order 3



The Two-Layer Perceptron



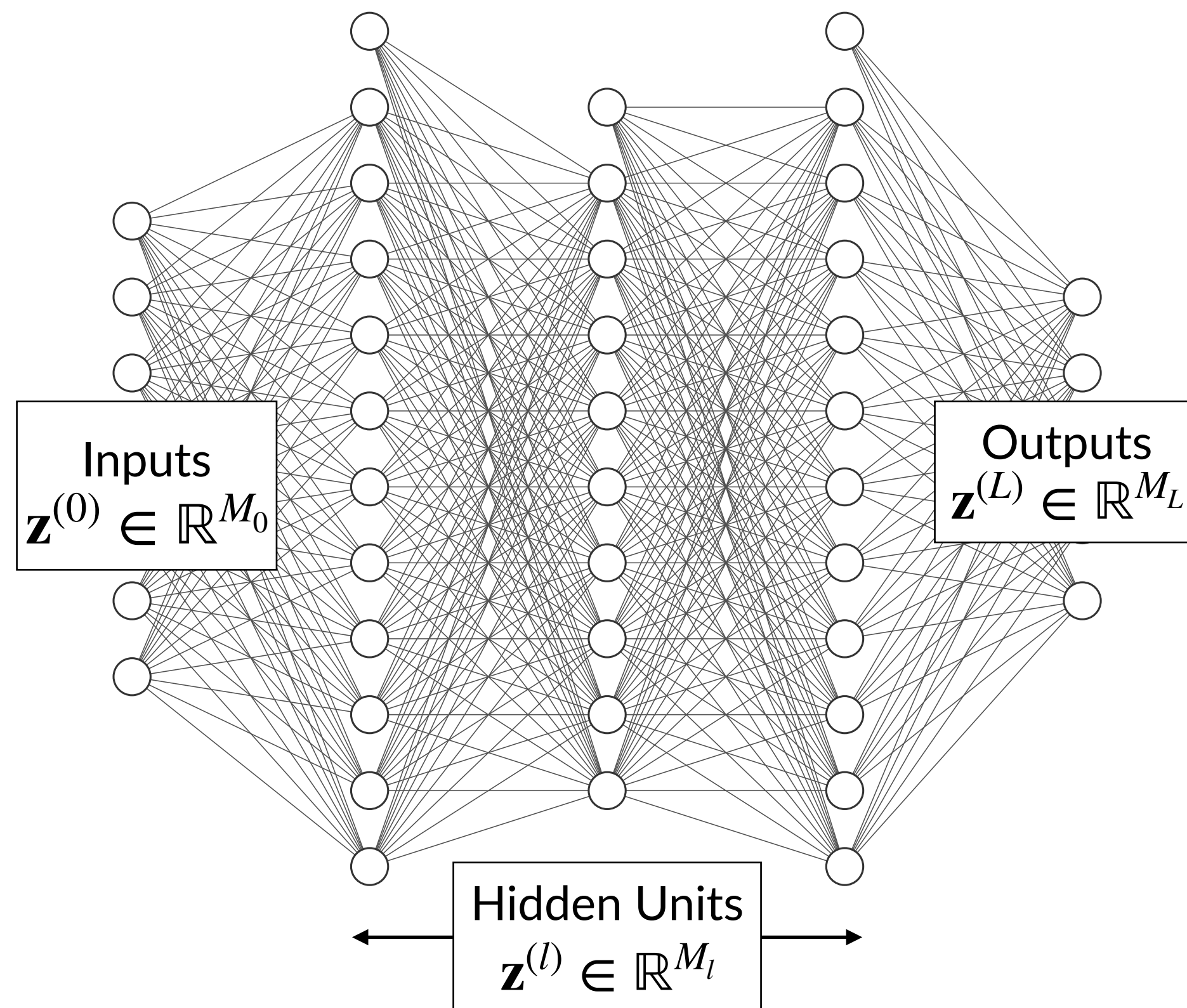
(1) $\longrightarrow$ (2)

$$y_k(\mathbf{x}, \mathbf{w}) = \underbrace{h^{(2)}}_{\text{Output-unit activation function}} \left(\sum_{j=0}^M w_{kj}^{(2)} \underbrace{h^{(1)}}_{\text{Hidden-unit activation function}} \left(\underbrace{\sum_{i=0}^D w_{ji}^{(1)} x_i}_{a_j^{(1)}} \right) \right)$$

$\underbrace{z_j^{(1)} = h^{(1)}(a_j^{(1)})}_{\text{Activation}}$

⚠ Activations $h^{(1)}(\cdot)$, $h^{(2)}(\cdot)$ applied to each vector element separately

Deep networks



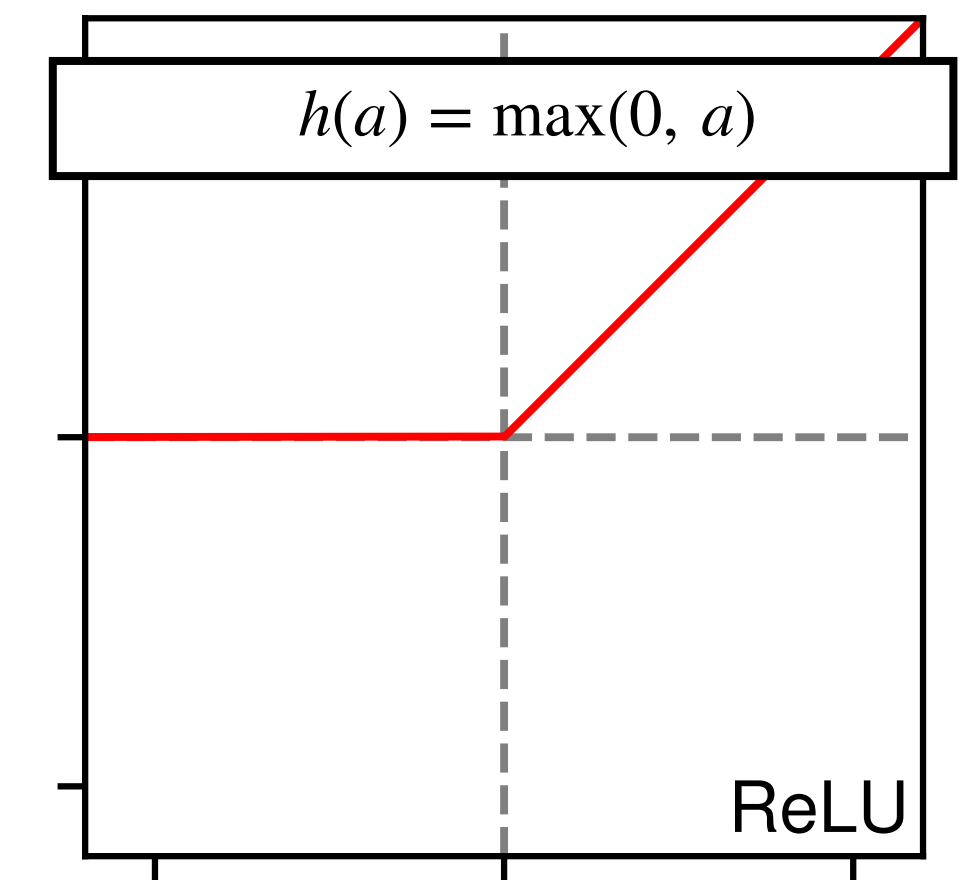
Layer:

Parametrised transformation applied uniformly across a collection of nodes. May include trainable parameters.

$$\forall l = 1, \dots, L, \quad \mathbf{z}^{(l)} = h^{(l)}(\mathbf{W}^{(l)} \mathbf{z}^{(l-1)})$$

Hidden Layer Activation function:

A non-linear function $h : \mathbb{R} \rightarrow \mathbb{R}$ applied element-wise to transform pre-activations into activations



Deep Learning in General

Feed-forward neural network:

A network where information flows in one direction through sequential transformations (**no cycles**)

$$z_k = h\left(\sum_{j \in \mathcal{A}(k)} w_{kj} z_j + b_k\right)$$

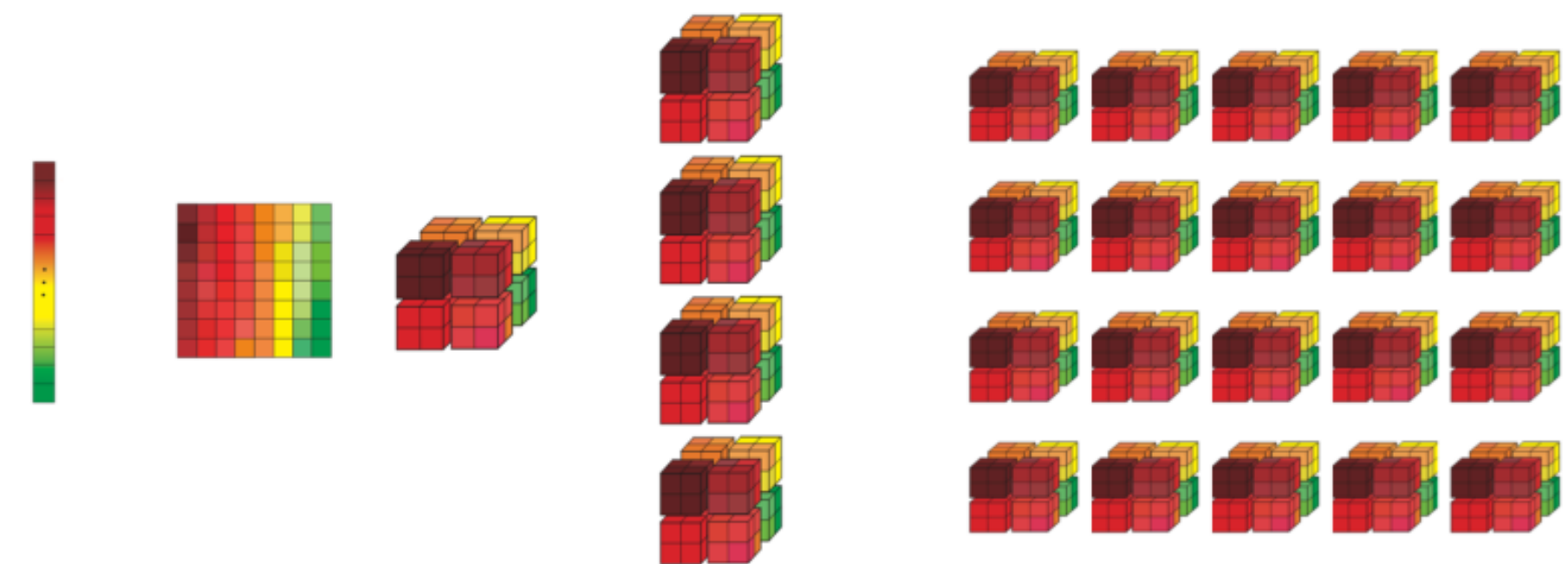
↑
Ancestors $\mathcal{A}$
of the node k

Tensors:

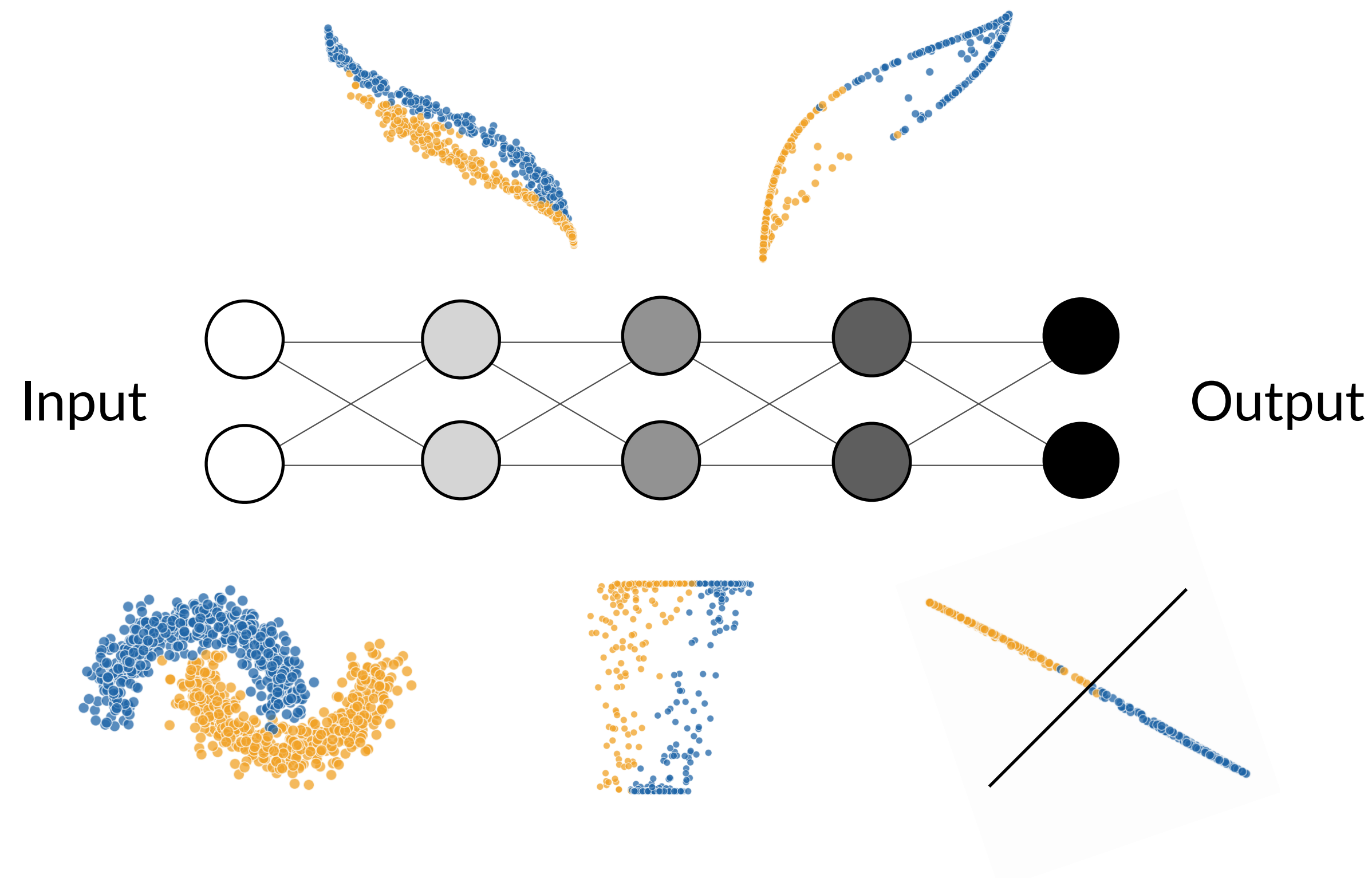
General data structure for Deep Learning

$$\mathbf{x} \in \mathbb{R}^{D_1 \times \dots \times D_K}$$

$$x_{i_1, i_2, \dots, i_K} \in \mathbb{R}, 1 \leq i_j \leq D_j \quad \forall j = 1, \dots, K$$



Representation Learning



Multi-Layer Perceptron with 3 hidden layers and a final output (linear) layer. Classification of two-moons dataset (Sci-kit learn).

Deep learning doesn't solve hard problems — it **learns representations** that make problems easy

Hidden layers automatically « discover » the relevant features

Next

Anatomy of a training step

Loss computation:

How far are our predictions from the true values?

TODAY

Backpropagation:

How should each parameter change to reduce the error?

LECTURE 5

Optimisation:

Update parameters in the direction that minimises the loss

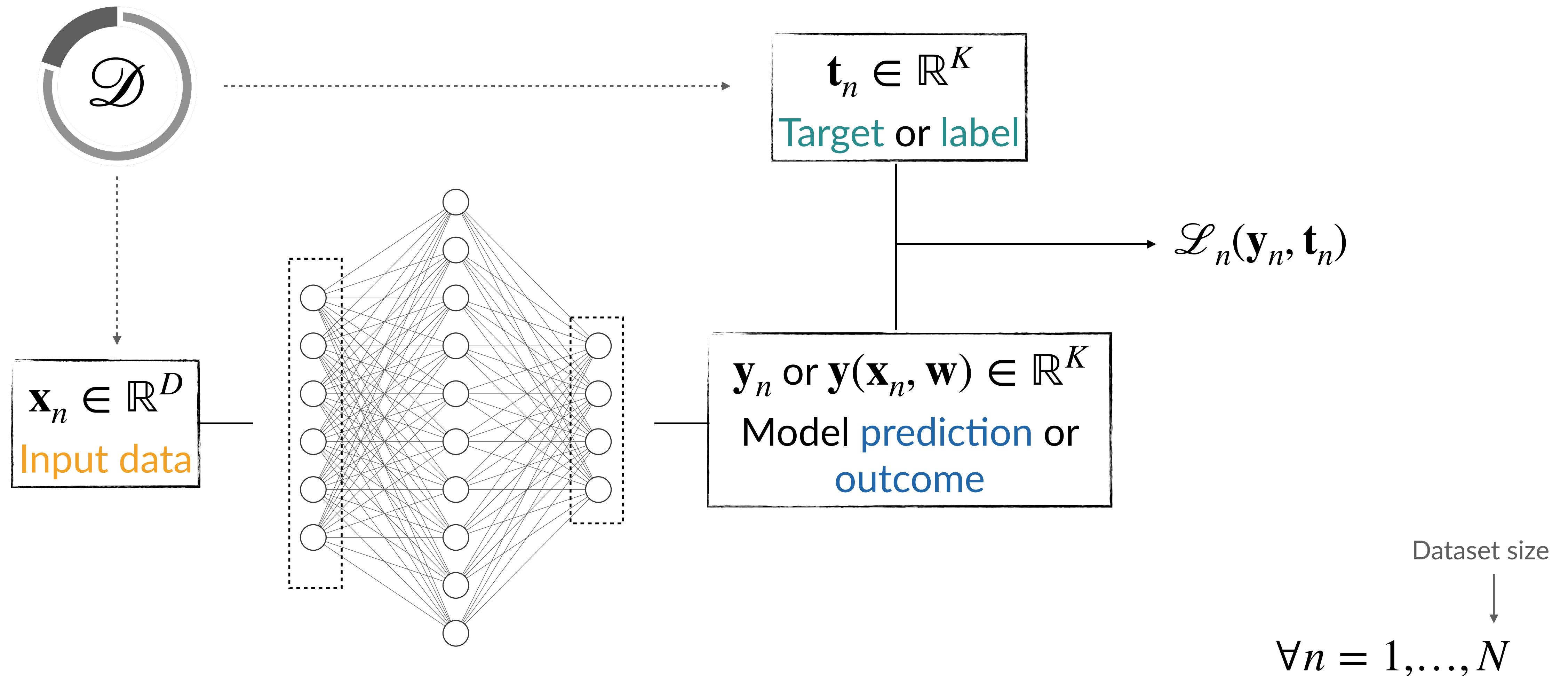
LECTURE 4

```
180 loss = criterion(outputs, y)
181 loss.backward()
182 optimiser.step()
```

MLP_classification.py

Losses

A note on notation



Loss functions

General strategy

● Probabilistic Model

Q: How likely is target $\mathbf{t}$, given input $\mathbf{x}$?

→ Assume

$$\mathbf{t} | \mathbf{x} \sim p(\cdot; \mathbf{y}(\mathbf{x}, \mathbf{w}))$$

↑
└ $\mathbf{t}$ depends on $\mathbf{x}$ through the
parametric model $\mathbf{y}(\mathbf{x}, \mathbf{w})$

See Bishop+ (2023), Sec. 6.4

Loss functions

General strategy

● Probabilistic Model

Q: How likely is target $\mathbf{t}$, given input $\mathbf{x}$?

→ Assume

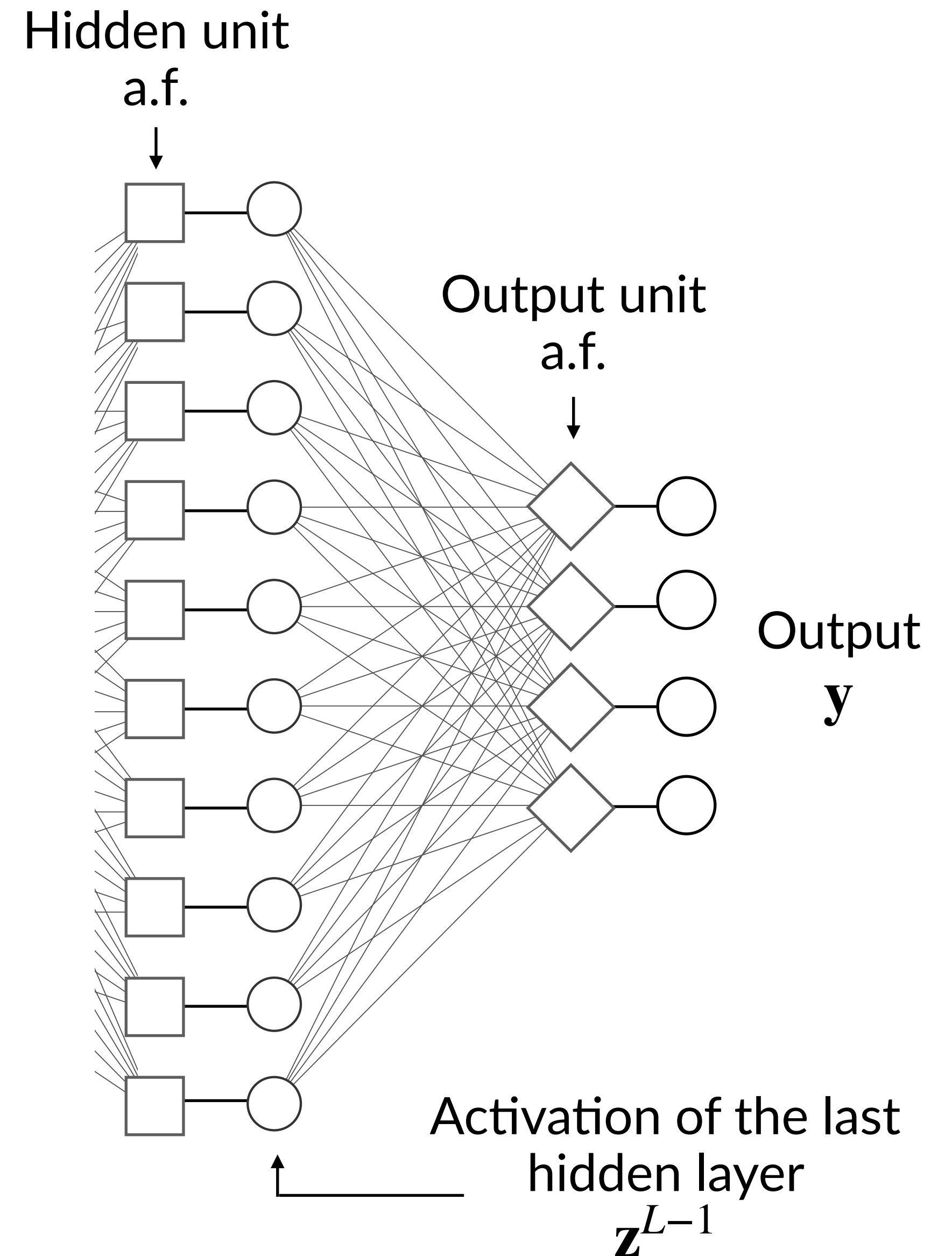
$$\mathbf{t} | \mathbf{x} \sim p(\cdot; \mathbf{y}(\mathbf{x}, \mathbf{w}))$$

Joint modelling
decisions

Q: What does $\mathbf{y}$ mean?

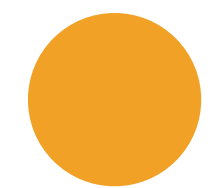
The « interpretation » of $\mathbf{y}$ dictates the
output-unit activation function

* Examples soon!



Loss functions

General strategy

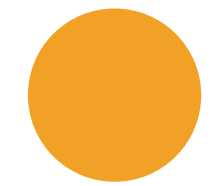


Probabilistic Model

Q: How likely is target $\mathbf{t}$, given input $\mathbf{x}$?

→ Assume

$$\mathbf{t} | \mathbf{x} \sim p(\cdot; \mathbf{y}(\mathbf{x}, \mathbf{w}))$$



Compute the **likelihood** of the dataset:

$$L_{\mathcal{D}}(\mathbf{w}) = \prod_{n=1}^N p(\mathbf{t}_n | \mathbf{x}_n, \mathbf{w})$$

Optimisation

Statistics: Maximum Likelihood

Optimal parameters maximise the Likelihood

$$\mathbf{w}^* = \arg \max_{\mathbf{w}} L_{\mathcal{D}}(\mathbf{w})$$

Loss functions

General strategy

- Probabilistic **Model**
Q: How likely is target $\mathbf{t}$, given input $\mathbf{x}$?
→ Assume
 $\mathbf{t} | \mathbf{x} \sim p(\cdot; \mathbf{y}(\mathbf{x}, \mathbf{w}))$
- Compute the **likelihood** of the dataset:
$$L_{\mathcal{D}}(\mathbf{w}) = \prod_{n=1}^N p(\mathbf{t}_n | \mathbf{x}_n, \mathbf{w})$$
- Define the **loss** as the negative log-likelihood
$$\mathcal{L}_{\mathcal{D}}(\mathbf{w}) = -\log L_{\mathcal{D}}(\mathbf{w})$$

Optimisation

Machine Learning: Loss minimisation

Optimal parameters minimise the Loss

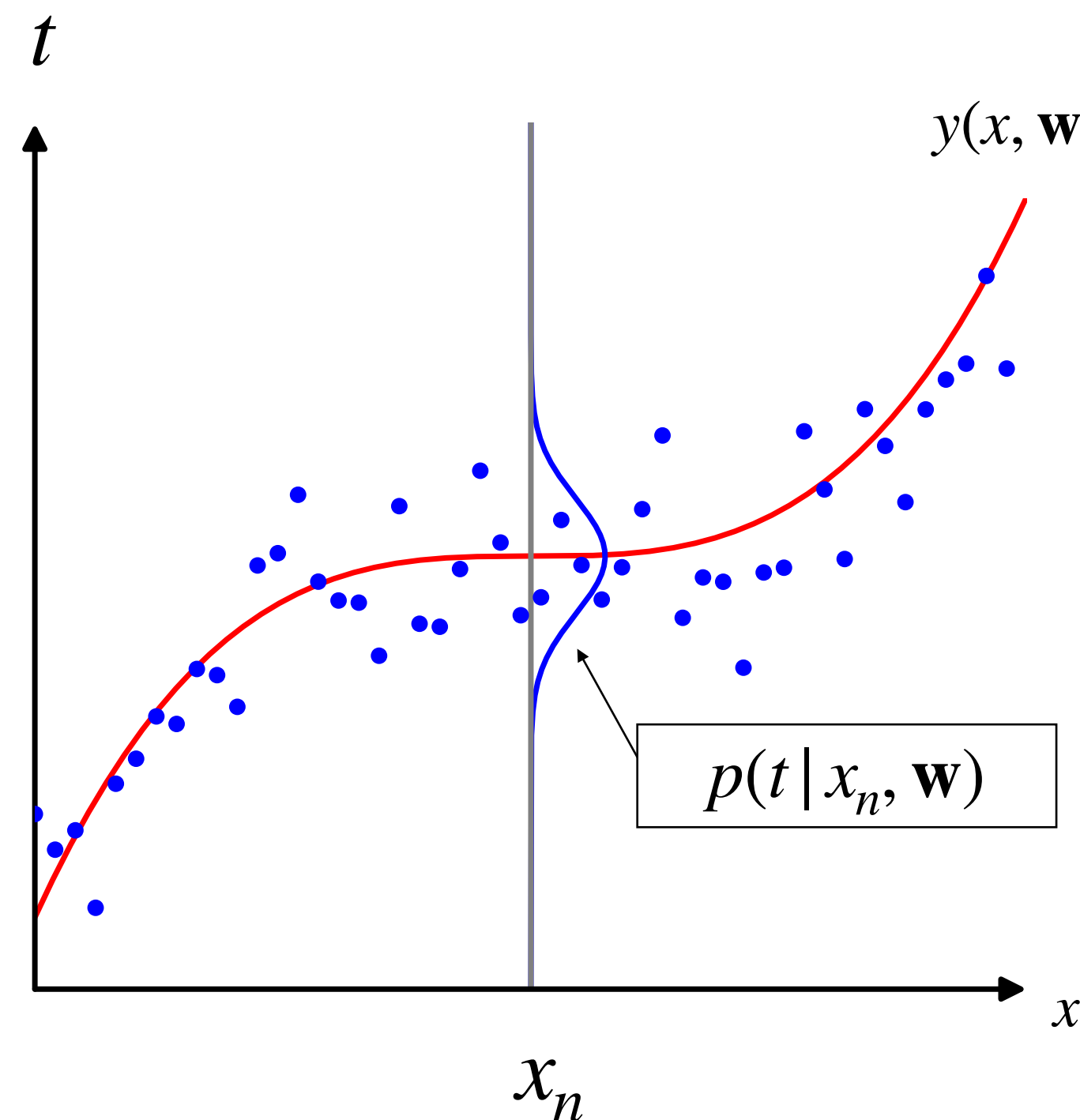
$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \mathcal{L}_{\mathcal{D}}(\mathbf{w})$$

Regression

Single target

● Probabilistic Model

$$p(t | \mathbf{x}, \mathbf{w}) = \mathcal{N}(t | y(\mathbf{x}, \mathbf{w}), \sigma^2) \sim \exp \left[-\frac{(y(\mathbf{x}, \mathbf{w}) - t)^2}{2\sigma^2} \right]$$

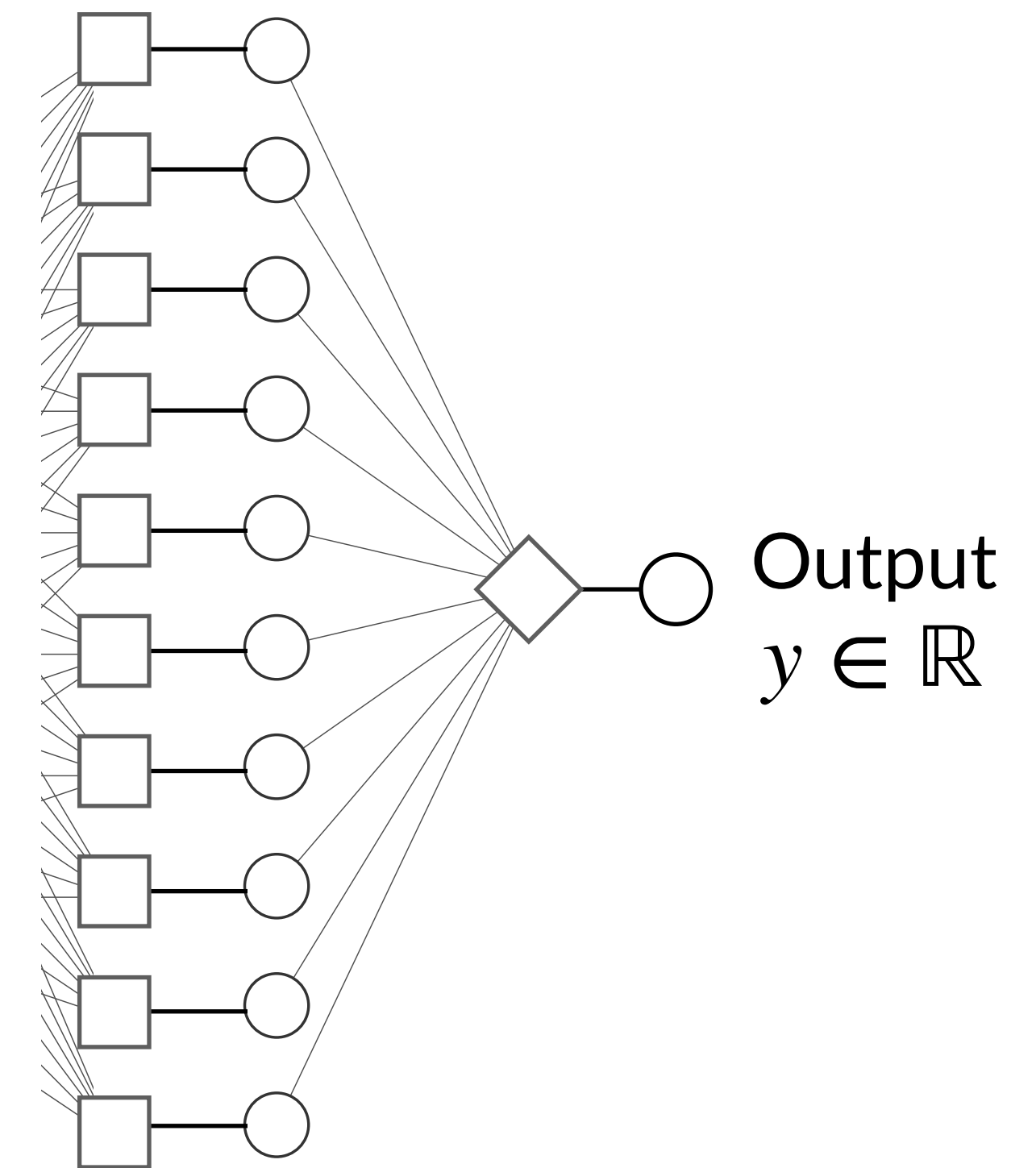


Interpretation of y : mean of the Gaussian distribution $\mathcal{N}$

Gaussian conditional distribution for $t \in \mathbb{R}$ given $x \in \mathbb{R}$. From Bishop+ (2023)

Gaussian distribution for $t \in \mathbb{R}$:

$$\mathcal{N}(t | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(t-\mu)^2}{2\sigma^2}}$$

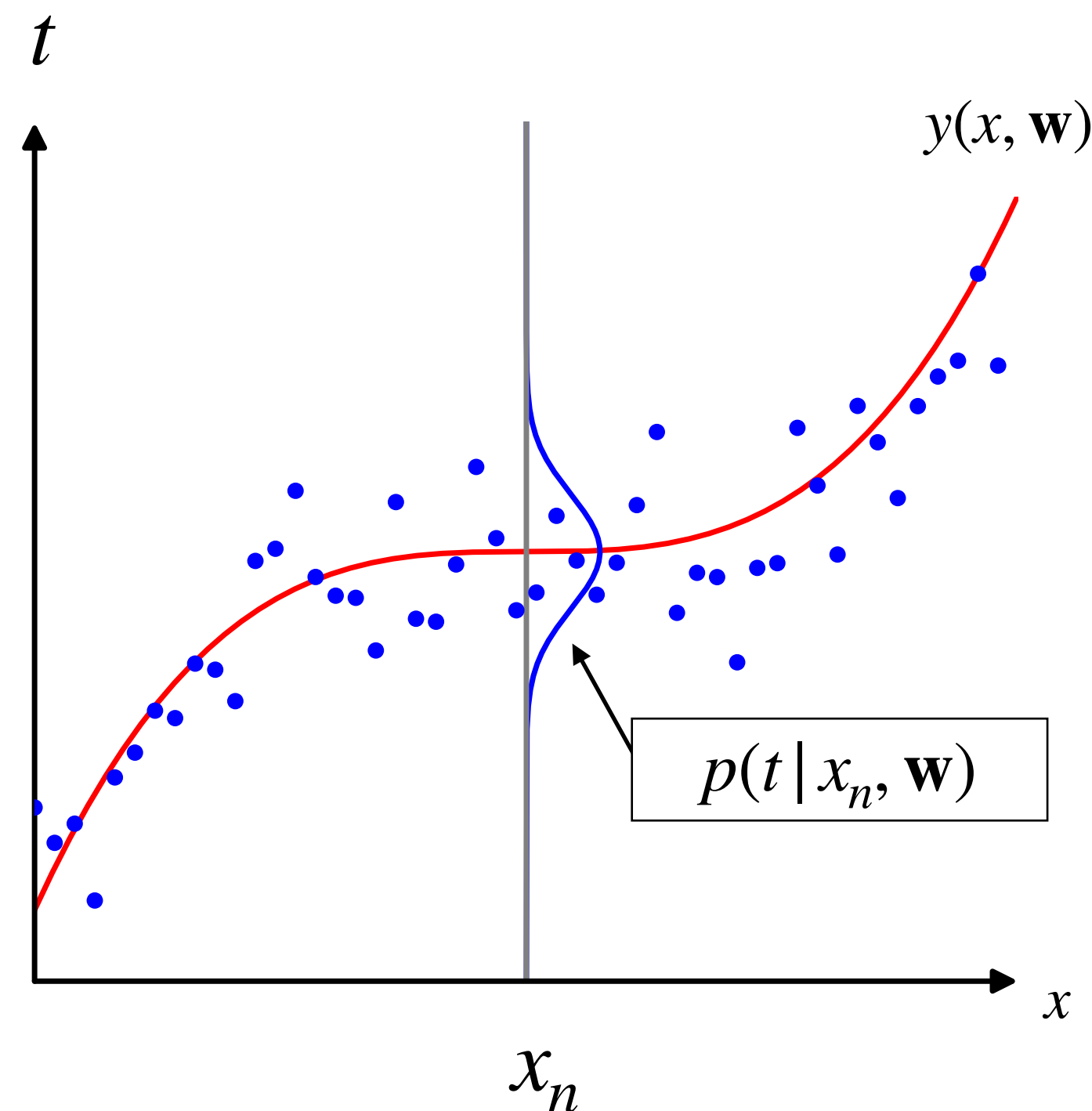


Regression

Single target

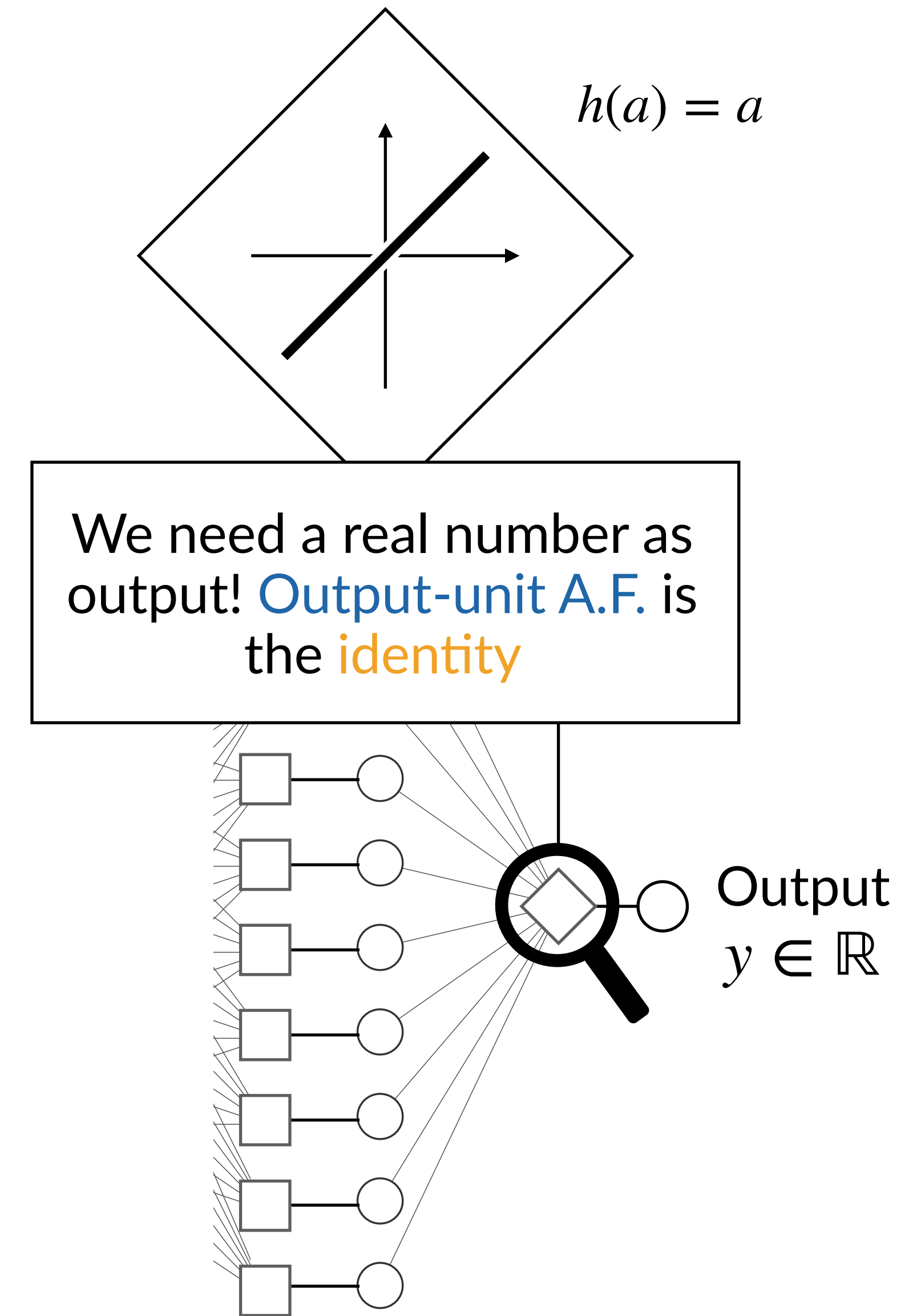
● Probabilistic Model

$$p(t | \mathbf{x}, \mathbf{w}) = \mathcal{N}(t | y(\mathbf{x}, \mathbf{w}), \sigma^2) \sim \exp \left[-\frac{(y(\mathbf{x}, \mathbf{w}) - t)^2}{2\sigma^2} \right]$$



Interpretation of y : mean of the Gaussian distribution $\mathcal{N}$

Gaussian conditional distribution for $t \in \mathbb{R}$ given $x \in \mathbb{R}$. From Bishop+ (2023)



Regression

Single target

- Probabilistic Model

$$p(t | \mathbf{x}, \mathbf{w}) = \mathcal{N}(t | y(\mathbf{x}, \mathbf{w}), \sigma^2) \sim \exp \left[-\frac{(y(\mathbf{x}, \mathbf{w}) - t)^2}{2\sigma^2} \right]$$

- Likelihood

$$L_{\mathcal{D}}(\mathbf{w}) = \prod_{n=1}^N \mathcal{N}(t_n | y(\mathbf{x}_n, \mathbf{w}), \sigma^2) \sim \exp \left[-\frac{\sum_{n=1}^N (y(\mathbf{x}_n, \mathbf{w}) - t_n)^2}{2\sigma^2} \right]$$

* Factors that do not depend on $\mathbf{w}$ have been omitted

Regression

Single target

● Probabilistic Model

$$p(t | \mathbf{x}, \mathbf{w}) = \mathcal{N}(t | y(\mathbf{x}, \mathbf{w}), \sigma^2) \sim \exp \left[-\frac{(y(\mathbf{x}, \mathbf{w}) - t)^2}{2\sigma^2} \right]$$

● Likelihood

$$L_{\mathcal{D}}(\mathbf{w}) = \prod_{n=1}^N \mathcal{N}(t_n | y(\mathbf{x}_n, \mathbf{w}), \sigma^2) \sim \exp \left[-\frac{\sum_{n=1}^N (y(\mathbf{x}_n, \mathbf{w}) - t_n)^2}{2\sigma^2} \right]$$

● Loss

$$\mathcal{L}_{\mathcal{D}}(\mathbf{w}) = -\log L(\mathbf{w}) \sim \frac{1}{2} \sum_{n=1}^N (y(\mathbf{x}_n, \mathbf{w}) - t_n)^2$$

* Factors that do not depend on $\mathbf{w}$ have been omitted

Regression

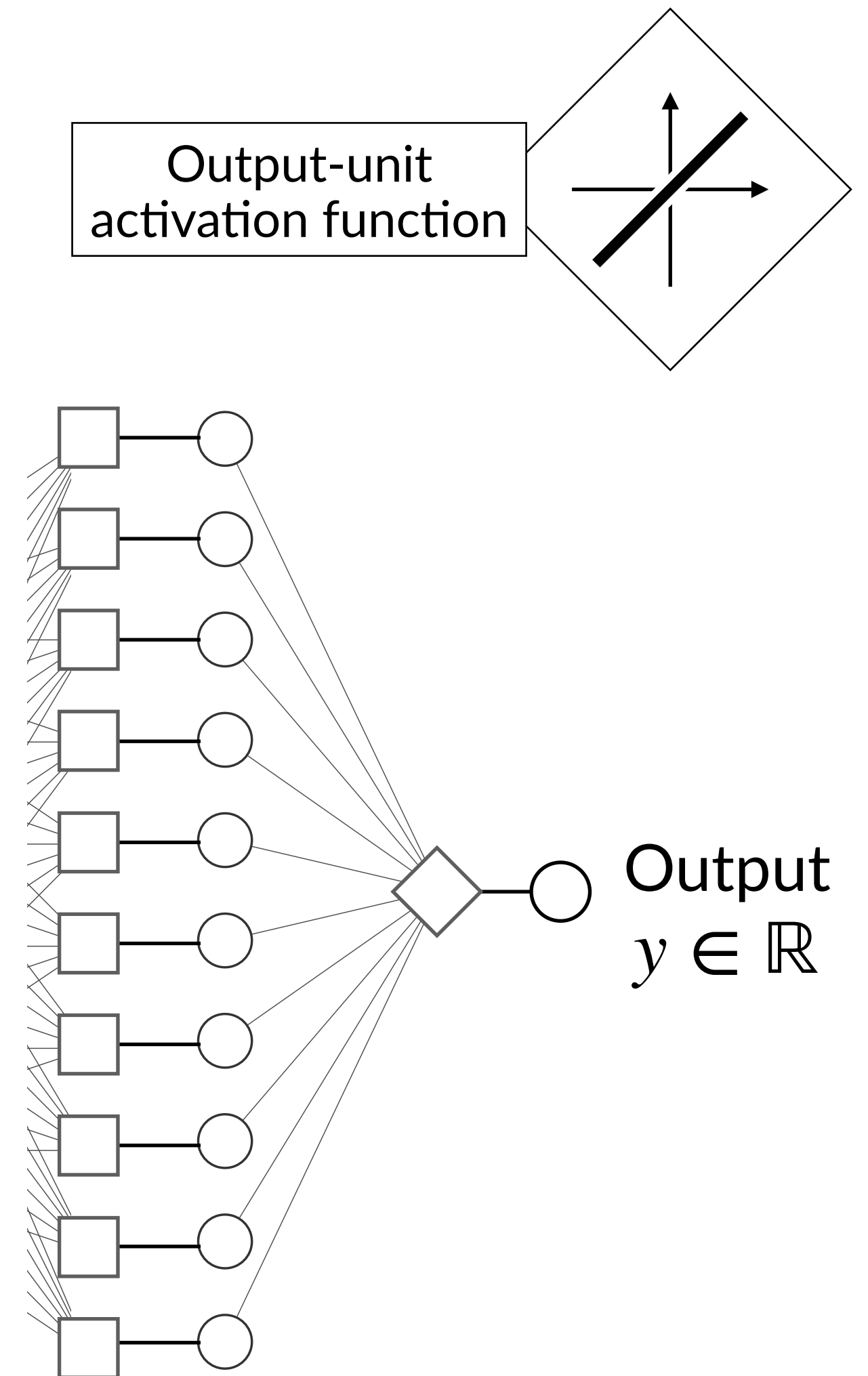
Single target

Sum of Squared Errors

$$\mathcal{L}_{\mathcal{D}}^{SSE}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \{y(\mathbf{x}_n, \mathbf{w}) - t_n\}^2$$

Because of the non-linearities in network function, highly complex — in particular, **non-convex** — function of the weights

```
import torch.nn as nn
criterion = nn.MSELoss(reduction='sum')
```



Regression

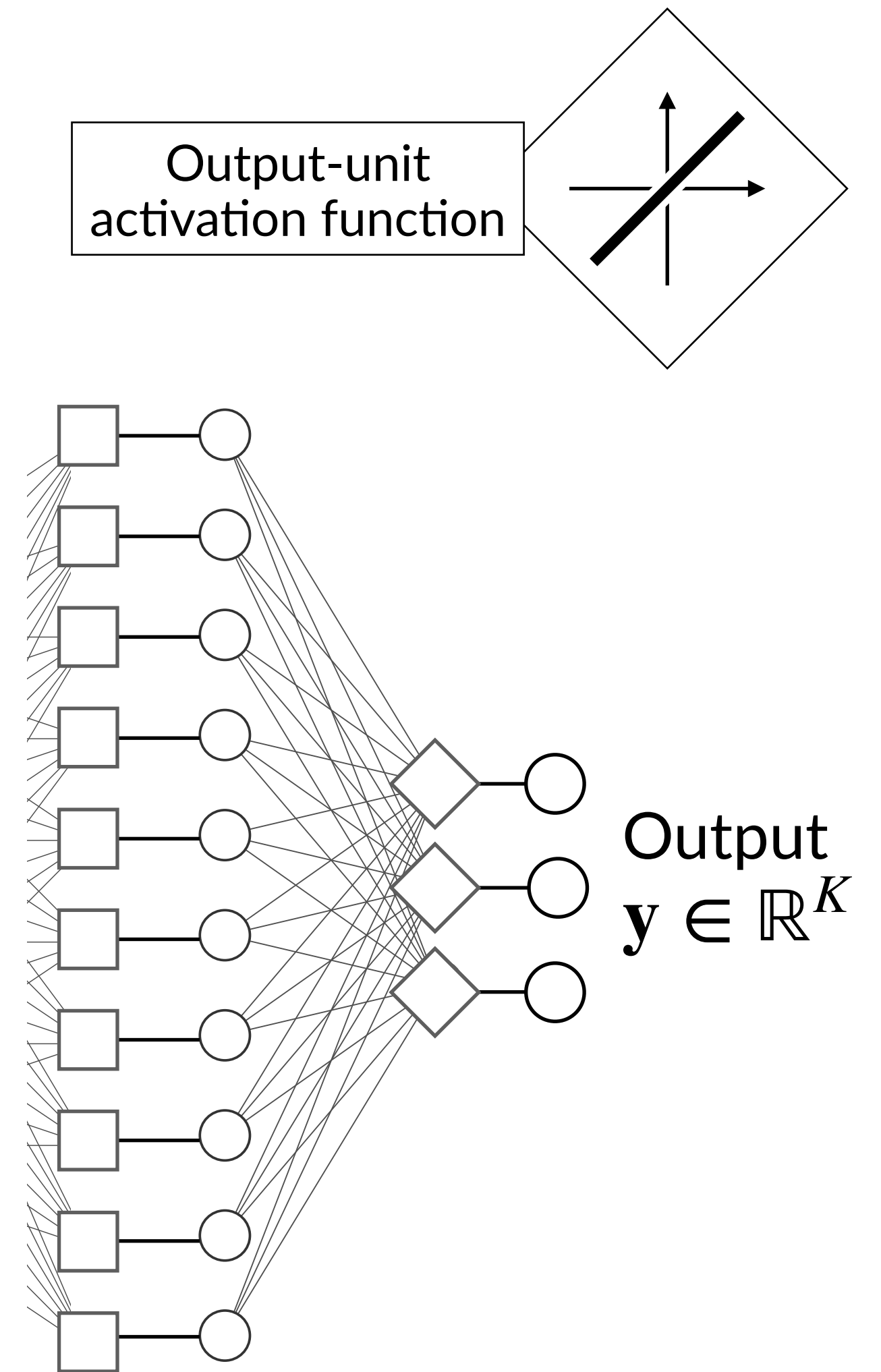
Multiple targets

Sum of Squared Errors

$$\mathcal{L}_{\mathcal{D}}^{SSE}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{t}_n\|^2$$

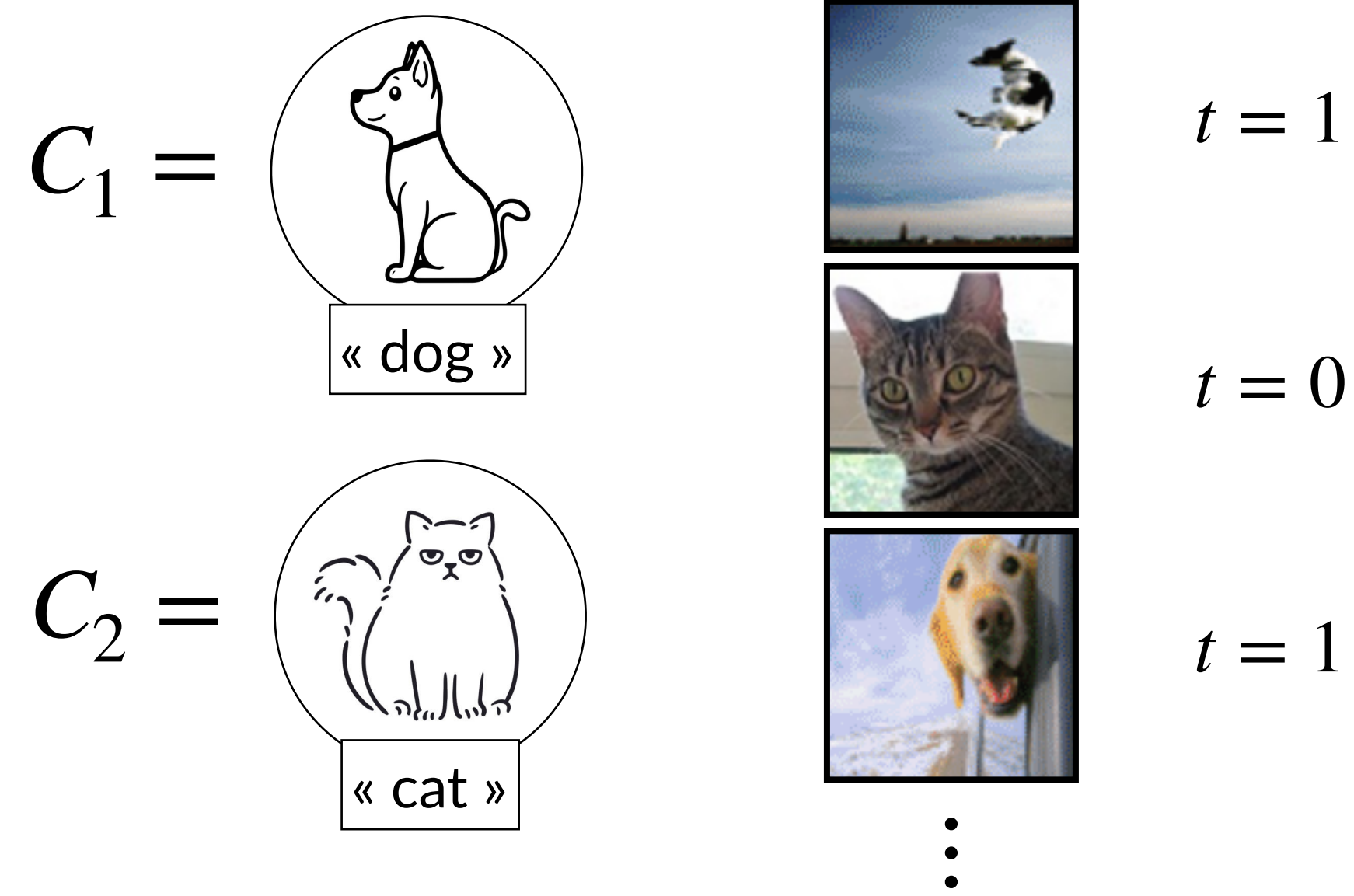
Because of the non-linearities in network function, highly complex — in particular, **non-convex** — function of the weights

```
import torch.nn as nn
criterion = nn.MSELoss(reduction='sum')
```

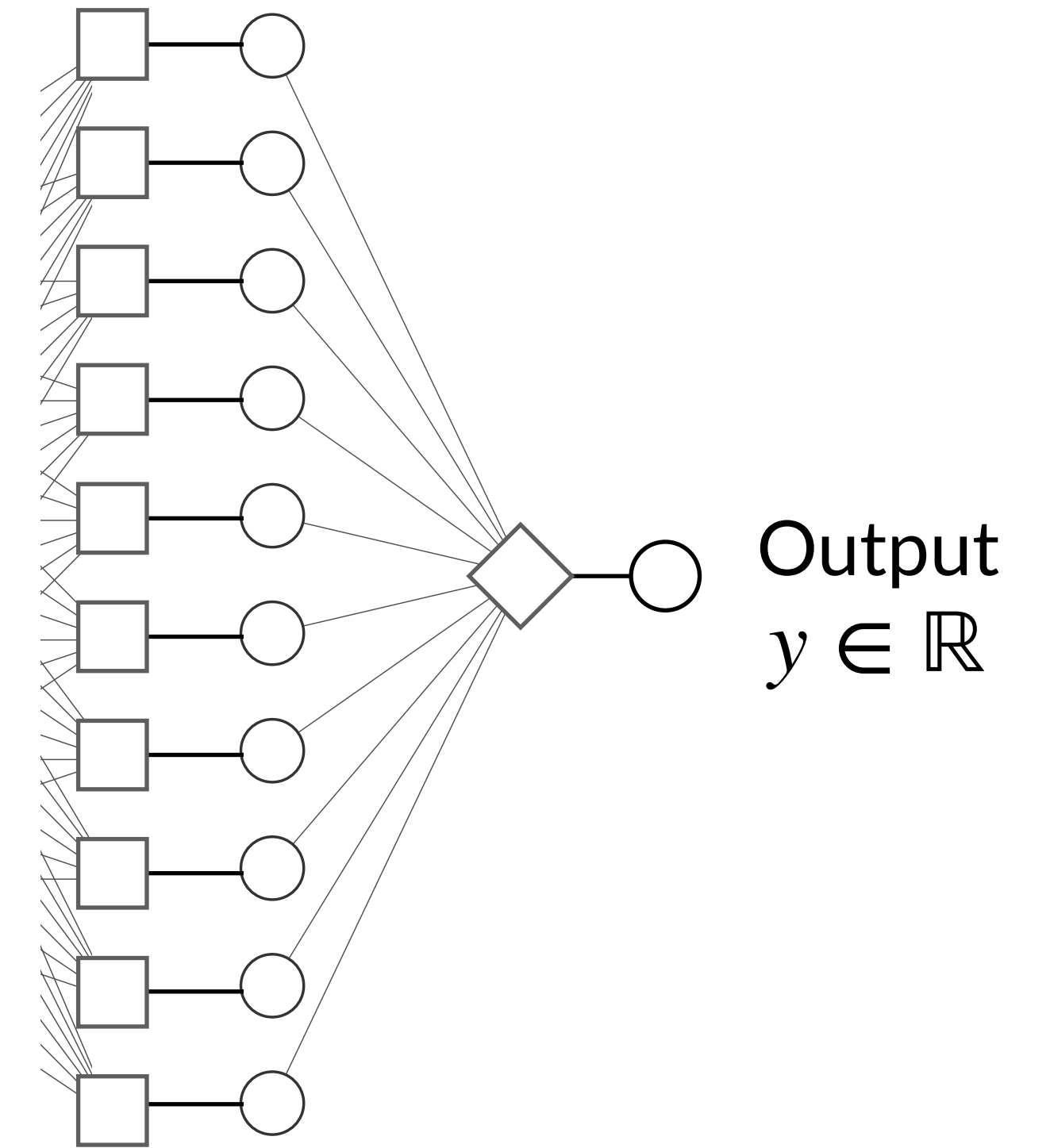


Binary Classification

Single target



Dogs and cats from the ImageNET dataset



Binary Classification

Single target

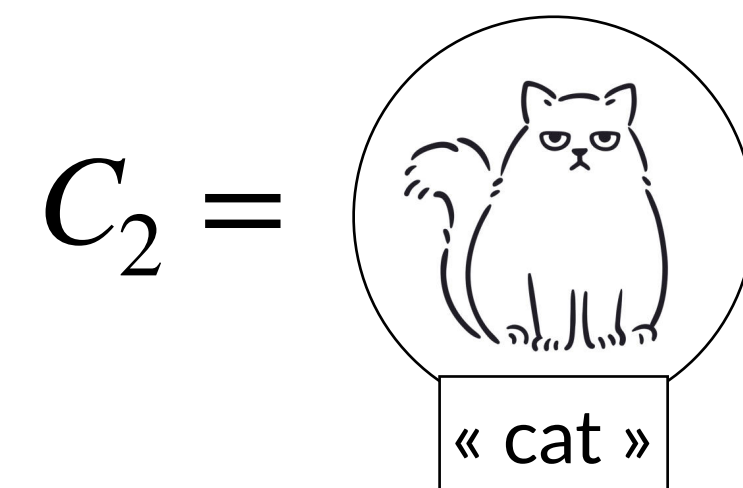
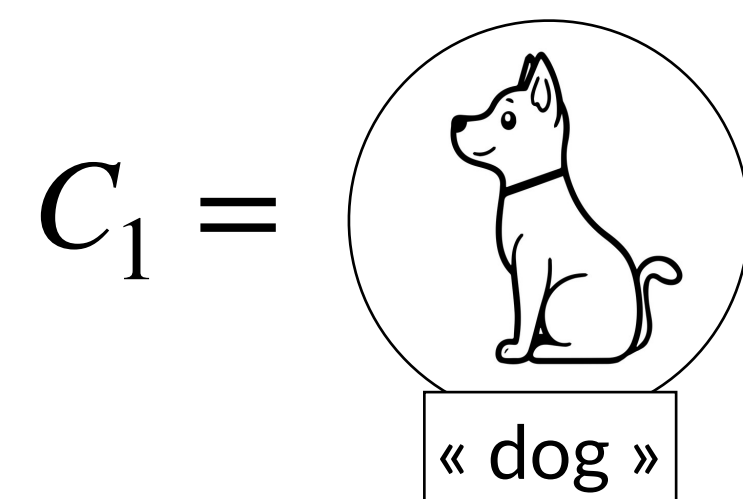
Bernoulli distribution for $t \in \{0,1\}$:

$$\text{Bern}(t | q) = q^t (1 - q)^{1-t}$$

In words, t takes the value 1 with probability q and 0 with probability $1 - q$

● Probabilistic Model

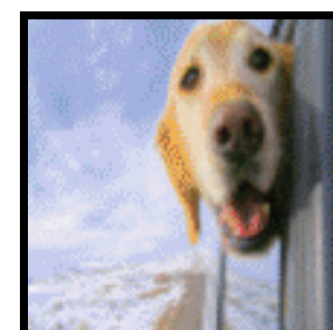
$$p(t | \mathbf{x}, \mathbf{w}) = \text{Bern}(t | y(\mathbf{x}, \mathbf{w})) = y(\mathbf{x}, \mathbf{w})^t \{1 - y(\mathbf{x}, \mathbf{w})\}^{1-t}$$



$t = 1$



$t = 0$



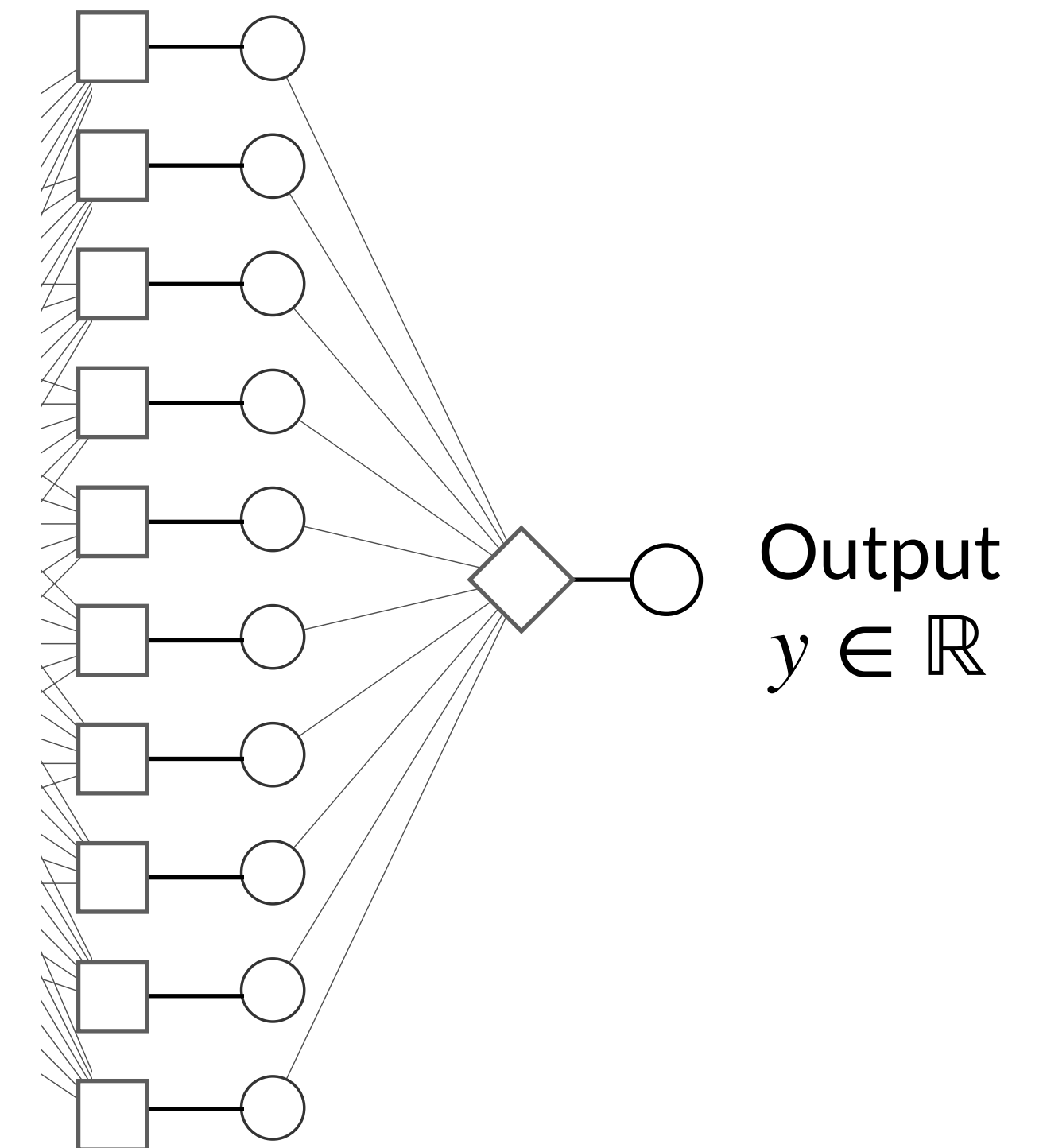
$t = 1$

⋮

Dogs and cats from the ImageNET dataset

Interpretation of y :
probability $p(C_1 | \mathbf{x})$ that $\mathbf{x}$
corresponds to the class C_1

$$\Rightarrow p(C_2 | \mathbf{x}) = 1 - y(\mathbf{x}, \mathbf{w})$$

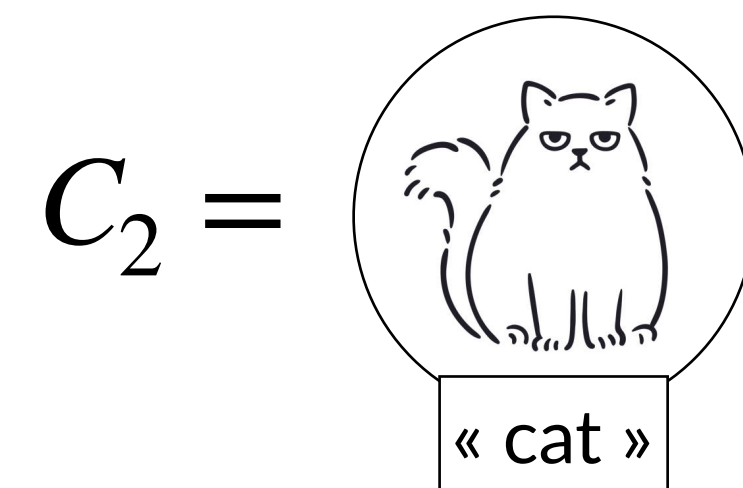
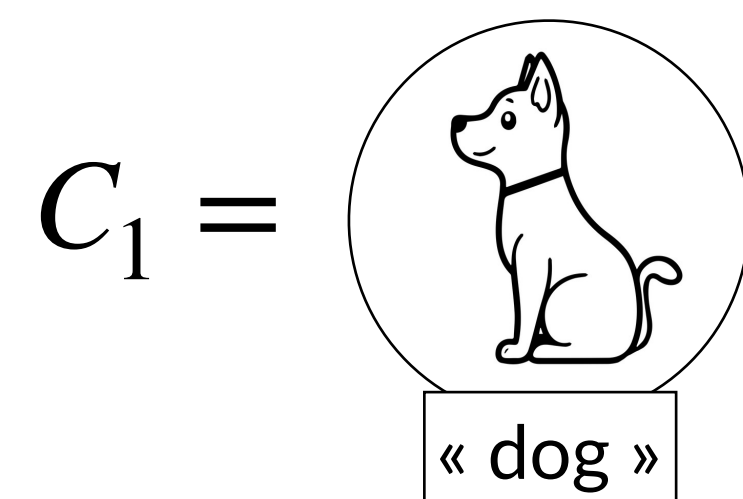


Binary Classification

Single target

● Probabilistic Model

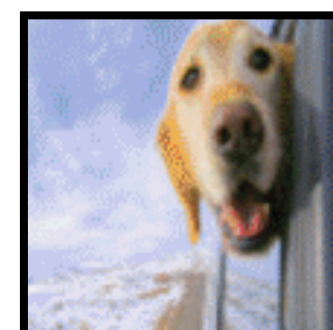
$$p(t | \mathbf{x}, \mathbf{w}) = \text{Bern}(t | y(\mathbf{x}, \mathbf{w})) = y(\mathbf{x}, \mathbf{w})^t \{1 - y(\mathbf{x}, \mathbf{w})\}^{1-t}$$



$t = 1$



$t = 0$



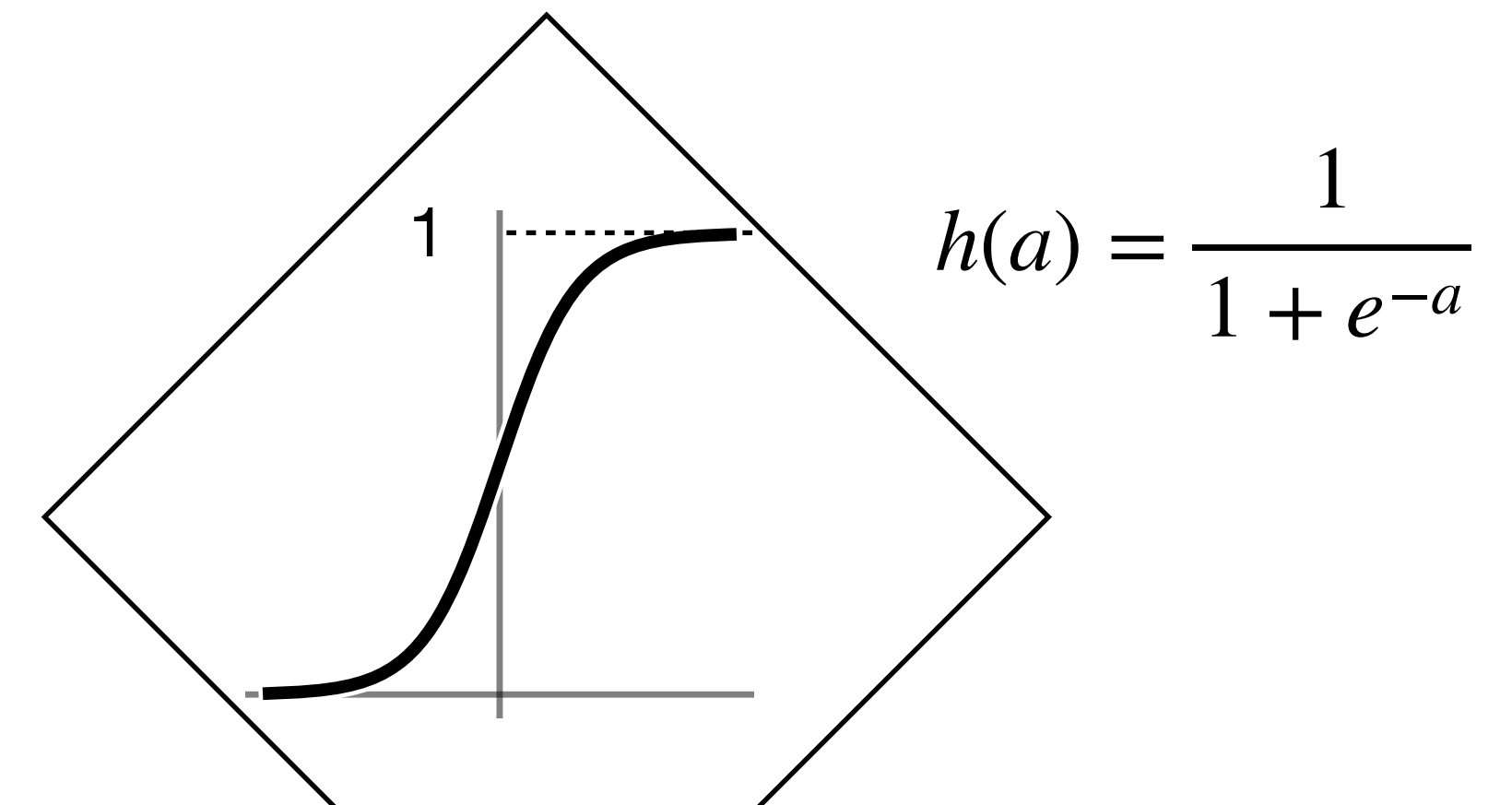
$t = 1$

⋮

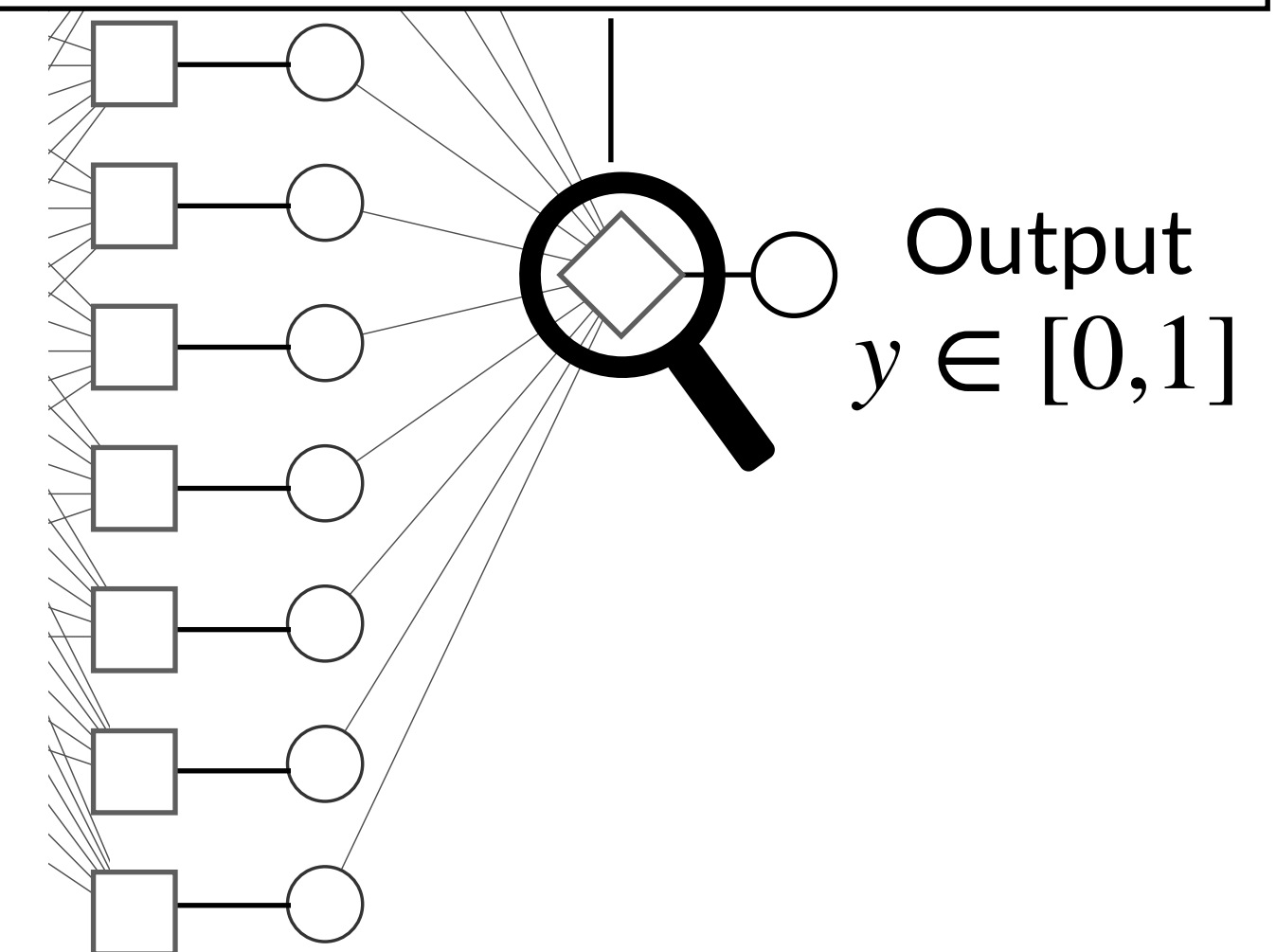
Dogs and cats from the ImageNET dataset

Interpretation of y :
probability $p(C_1 | \mathbf{x})$ that $\mathbf{x}$
corresponds to the class C_1

$$\Rightarrow p(C_2 | \mathbf{x}) = 1 - y(\mathbf{x}, \mathbf{w})$$



We need a real number between 0 and 1 as output! **Output-unit A.F.** is the **logistic sigmoid**



Binary Classification

Single target

Binary cross-entropy

$$\mathcal{L}_{\mathcal{D}}^{BCE}(\mathbf{w}) = - \sum_{n=1}^N \{ t_n \log y_n - (1 - t_n) \log(1 - y_n) \}$$

$$^* y_n = y(\mathbf{x}_n, \mathbf{w})$$

Why « cross entropy »?

Measures the distance between the true distribution and the predicted distribution

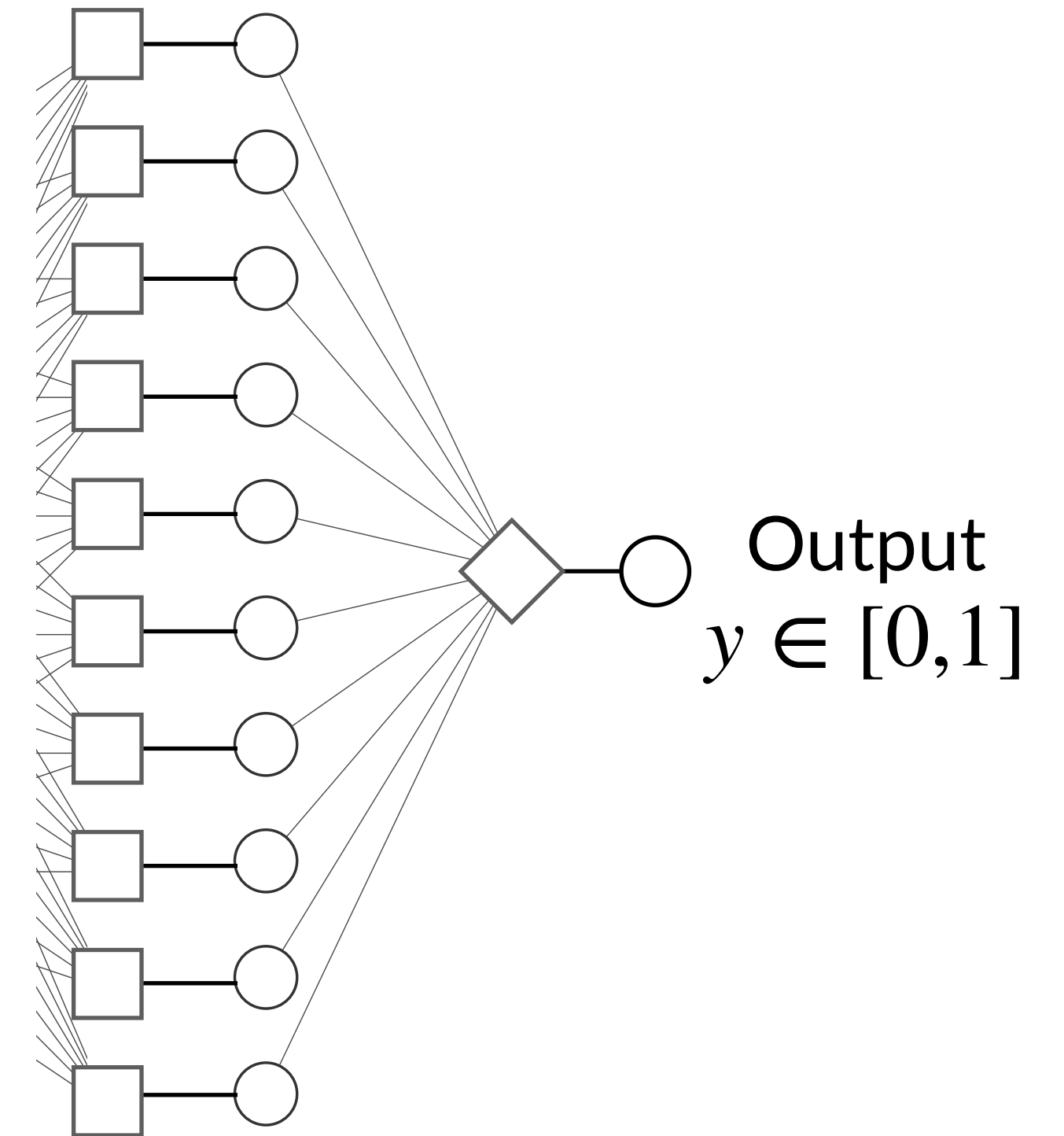
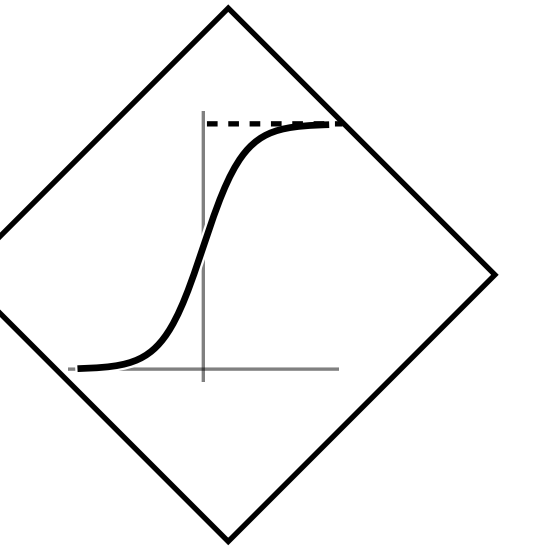
$$H(p, q) = H(p) + D_{KL}(p \| q) = \sum_{s \in \mathcal{S}} p(s) \log q(s)$$

Bishop+ (2023), Sec. 2.5

Thomas+ (1999), Ch. 2

```
import torch.nn as nn
criterion = nn.BCEWithLogitsLoss()
```

Output-unit
activation function



Binary Classification

Multiple targets

Binary cross-entropy

$$\mathcal{L}_{\mathcal{D}}^{BCE}(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^{M_L} \left\{ t_{nk} \log y_{nk} - (1 - t_{nk}) \log(1 - y_{nk}) \right\}$$

$* y_{nk} = y_k(\mathbf{x}_n, \mathbf{w})$



Classifiers

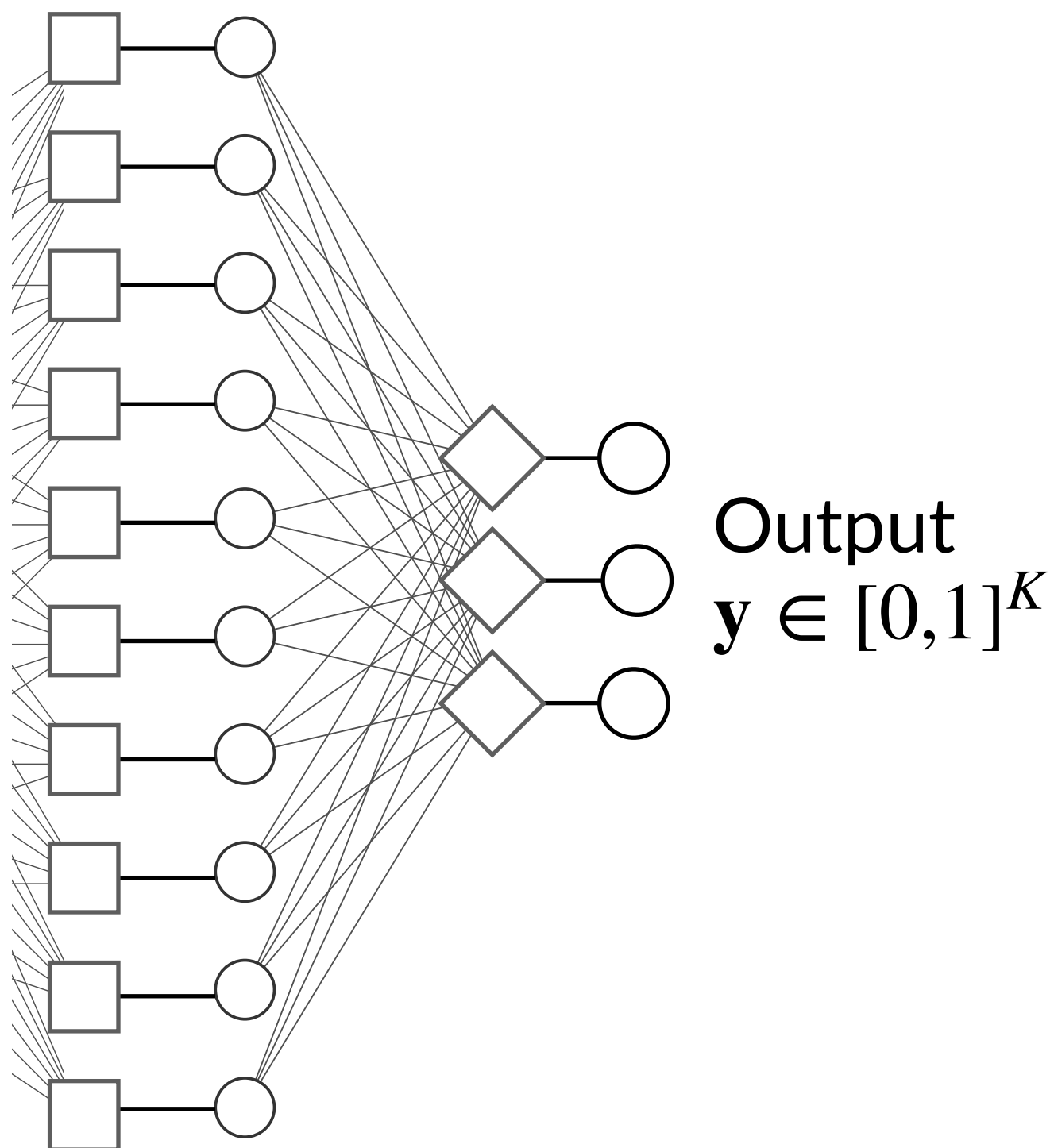
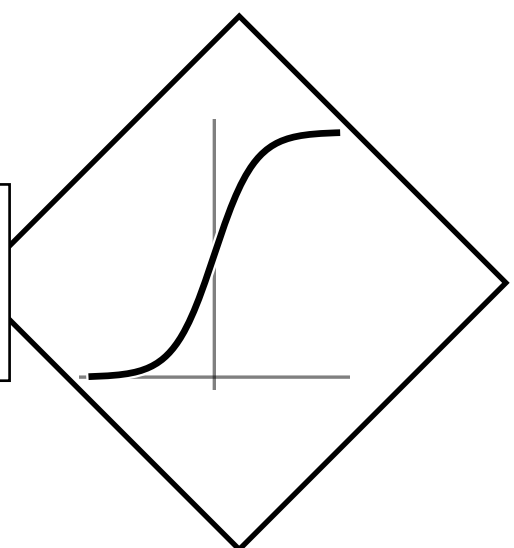
$k = 1$ / Dog (C_1) or cat (C_2)?
 $k = 2$ / Long-hair (C_1) or short-hair (C_2) ?
 $k = 3$ / Airborne (C_1) or grounded (C_2)?
 $\vdots$

Label

$t_{n1} = 1$
 $t_{n2} = 0$
 $t_{n3} = 1$
 $\vdots$

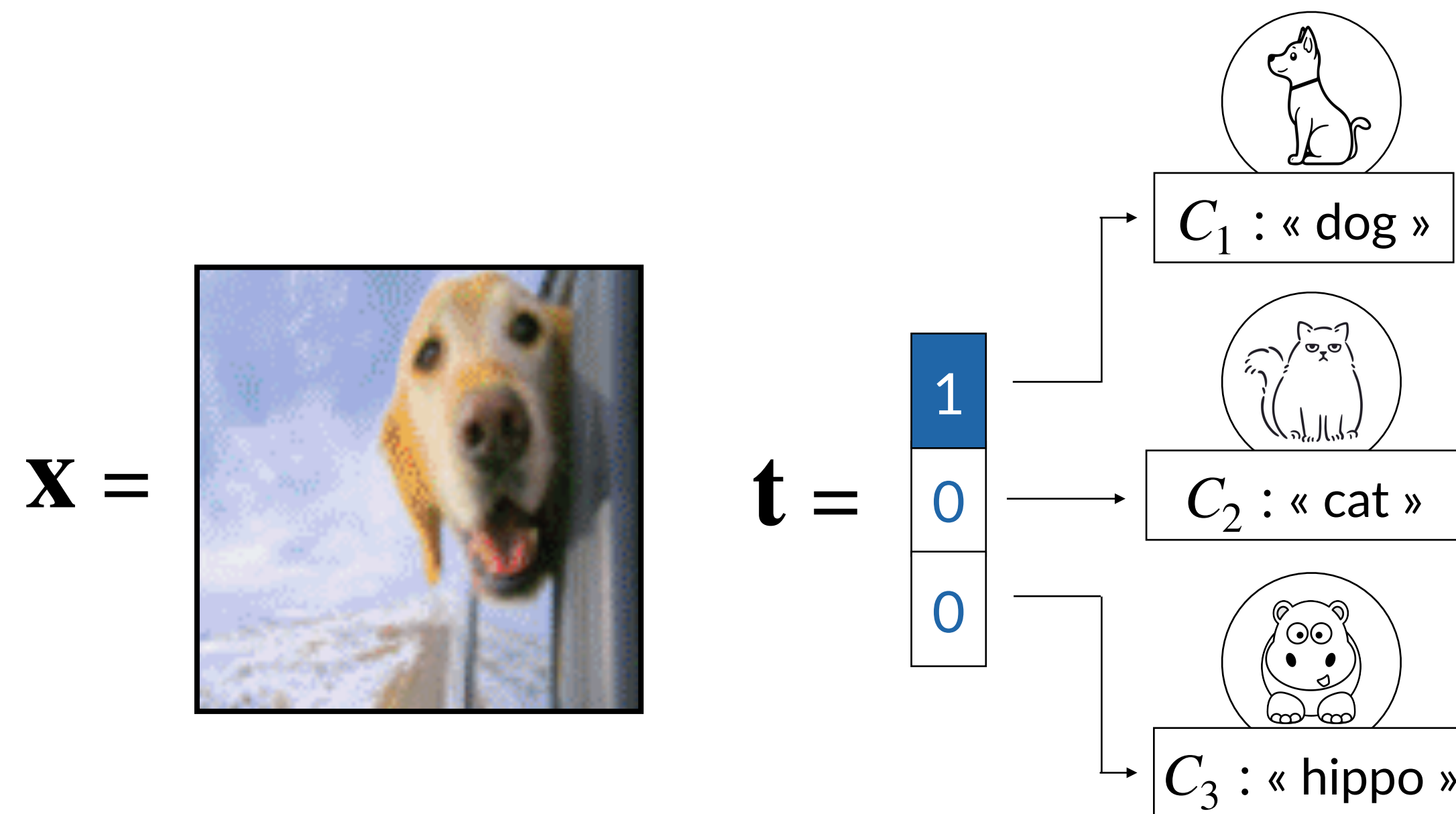
1
0
1
 $\vdots$

Output-unit
activation function



Multi-class Classification

One-hot encoding scheme



One-hot (or 1-of-K) encoding scheme:

If the **true class** is C_j then

$$t_k = \begin{cases} 0 & \text{if } k \neq j \\ 1 & \text{if } k = j \end{cases}$$

We can interpret t_k as « probability of the class C_k »:
only one class has probability one, the rest has probability zero!

$$\sum_k t_k = 1$$

Multi-class Classification

Probabilistic model

● Probabilistic Model

$$p(\mathbf{t} | \mathbf{x}, \mathbf{w}) = \prod_{k=1}^K y_k(\mathbf{x}, \mathbf{w})^{t_k}$$

Interpretation of $y_k(\mathbf{x}, \mathbf{w})$:
probability $p(C_k | \mathbf{x})$ that $\mathbf{x}$
corresponds to the class C_k

$\mathbf{x} =$



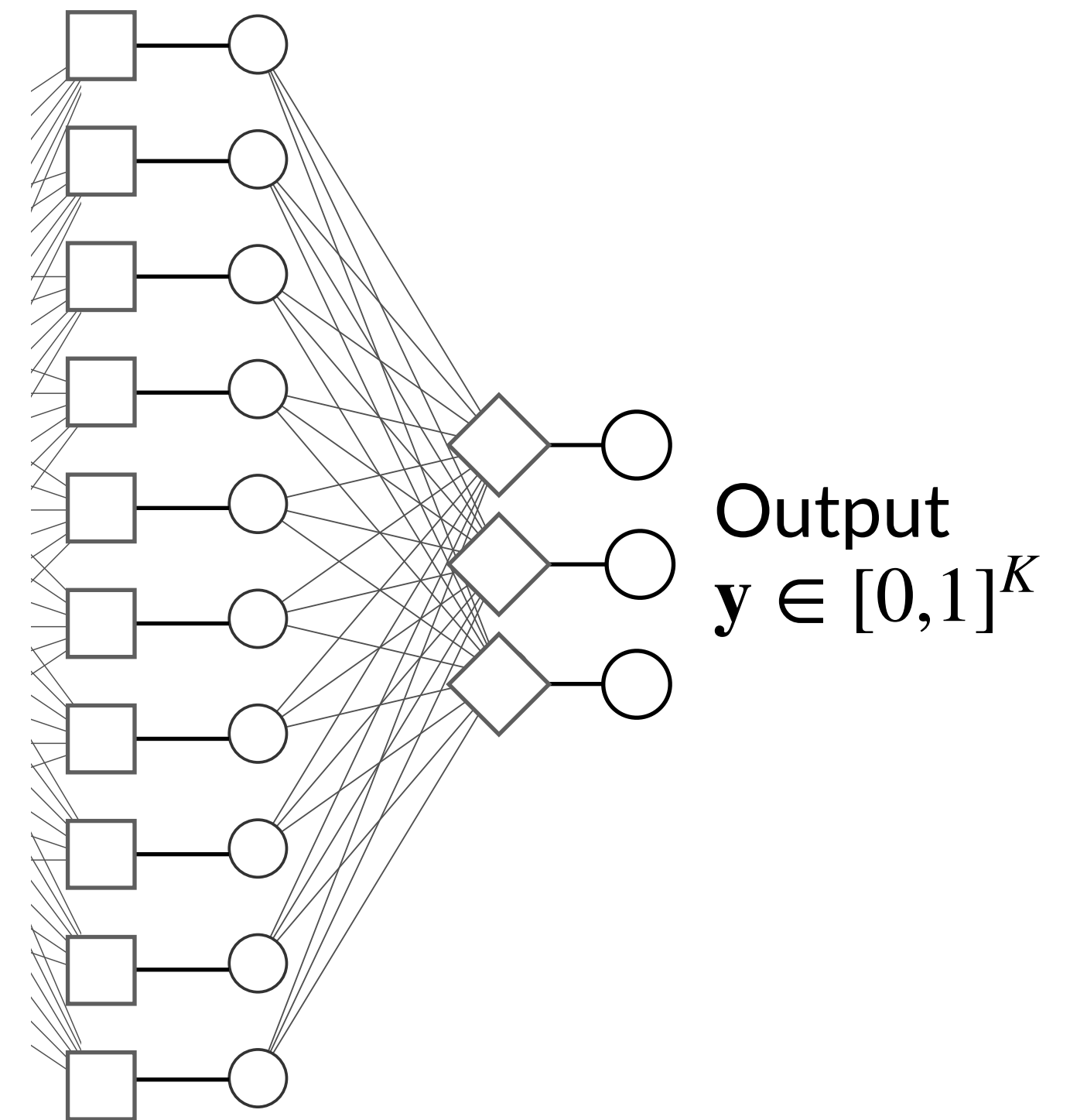
$\mathbf{t} =$

1
0
0

To be consistent, we need
 $y_k(\mathbf{x}, \mathbf{w}) \in [0,1] \ \forall k$

but also

$$\sum_k y_k(\mathbf{x}, \mathbf{w}) = 1$$



Multi-class Classification

Softmax activation function

Output-unit activation function is

Softmax

$$y_k(\mathbf{x}, \mathbf{w}) = \frac{\exp(a_k^{(L)}(\mathbf{x}, \mathbf{w}))}{\sum_j \exp(a_j^{(L)}(\mathbf{x}, \mathbf{w}))}$$

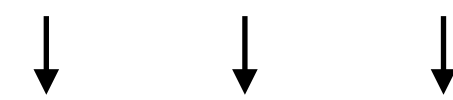
 Not applied element-wise:
outputs are coupled

Final pre-activations or **logits**
(unnormalised probabilities)

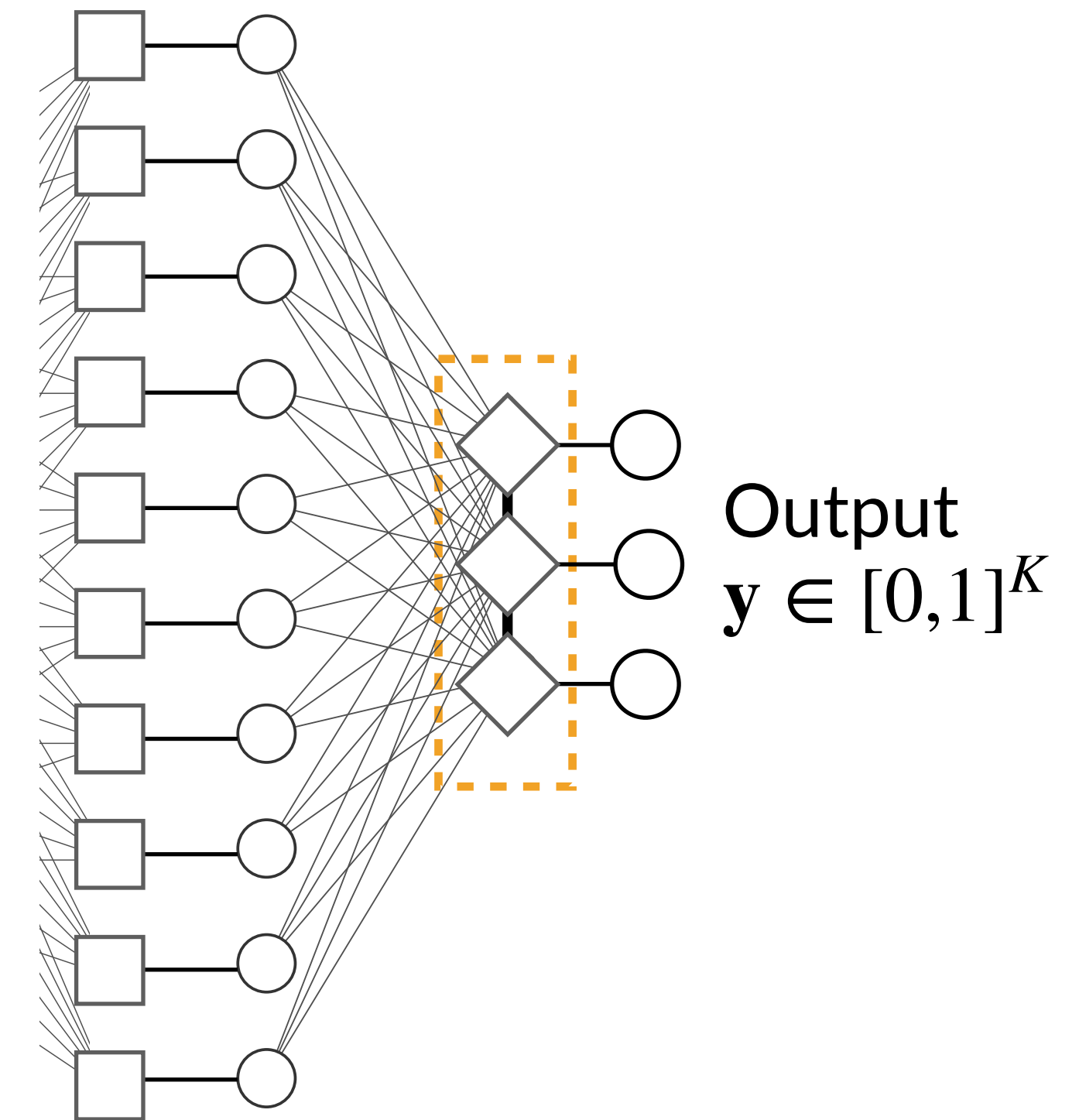
$$a_j = \sum_{i=1}^{M_{L-1}} w_{ji}^{(L)} z_i^{(L-1)}$$

Example:

$$\mathbf{a} = [2.10, 0.52, 1.25]$$



$$\mathbf{y} = [0.61, 0.13, 0.26]$$



Multi-class Classification

Cross-entropy

$$\mathcal{L}_{\mathcal{D}}^{CE}(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log y_{nk}$$

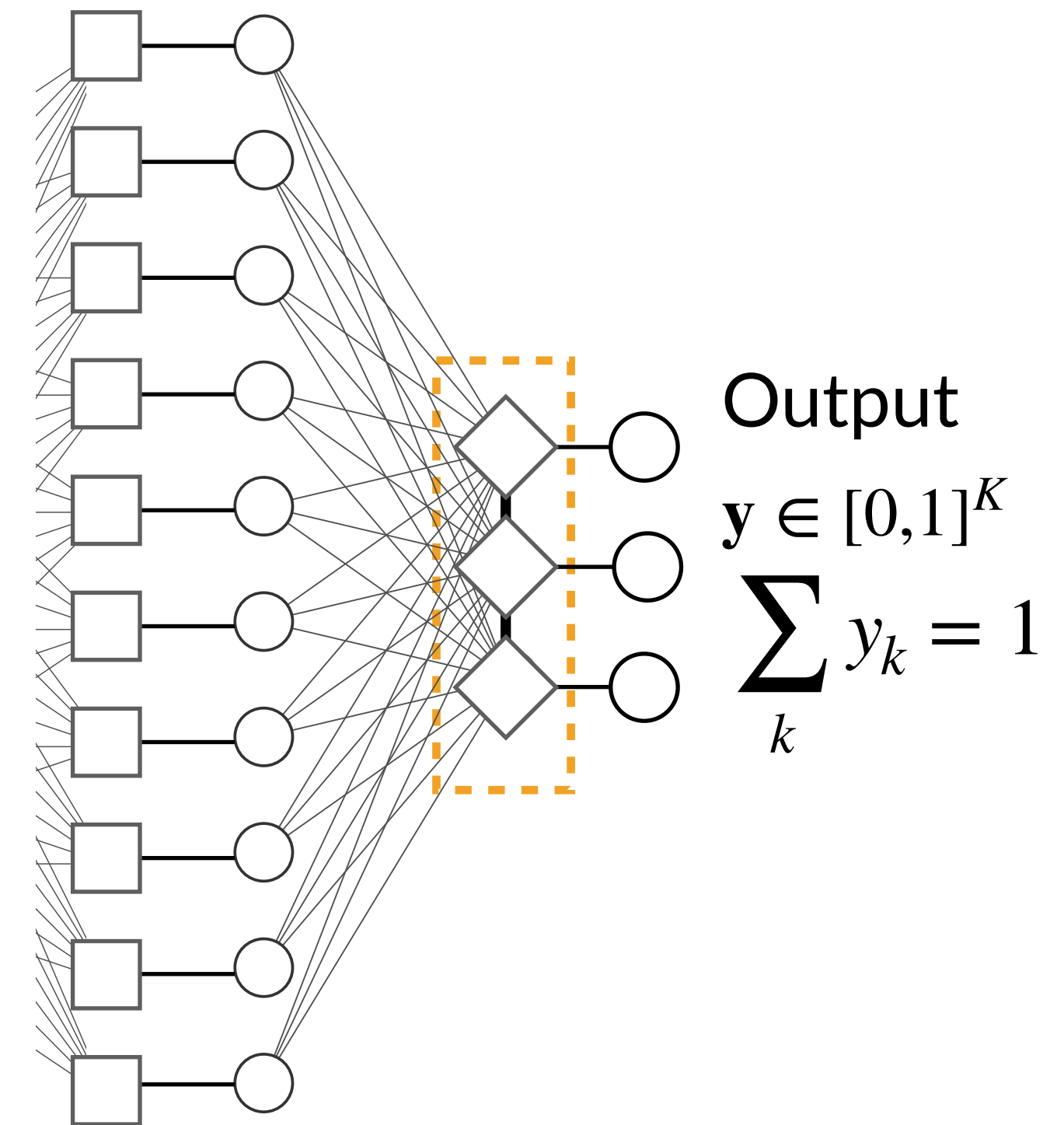
* $y_{nk} = y_k(\mathbf{x}_n, \mathbf{w})$

Standard loss for classification:
encourages high probability for the correct
class

```
import torch.nn as nn  
criterion = nn.CrossEntropyLoss()
```

Output-unit
activation function

Softmax: $y_k = \frac{\exp(a_k^{(L)})}{\sum_j \exp(a_j^{(L)})}$



Loss functions

Summary

The loss function $\mathcal{L}$ quantifies **how wrong our predictions are**

It's what we minimise during training

Loss functions derived from probabilistic models: assumptions, losses, and output activations.
See Bishop+ (2023), Sec. 6.4

- ✓ **General recipe** for standard losses
Probabilistic model $\rightarrow$ Likelihood of the dataset $\rightarrow$ Loss
- ✓ **Loss $\leftrightarrow$ Activation**
Probabilistic assumption determines both the loss and the output activation

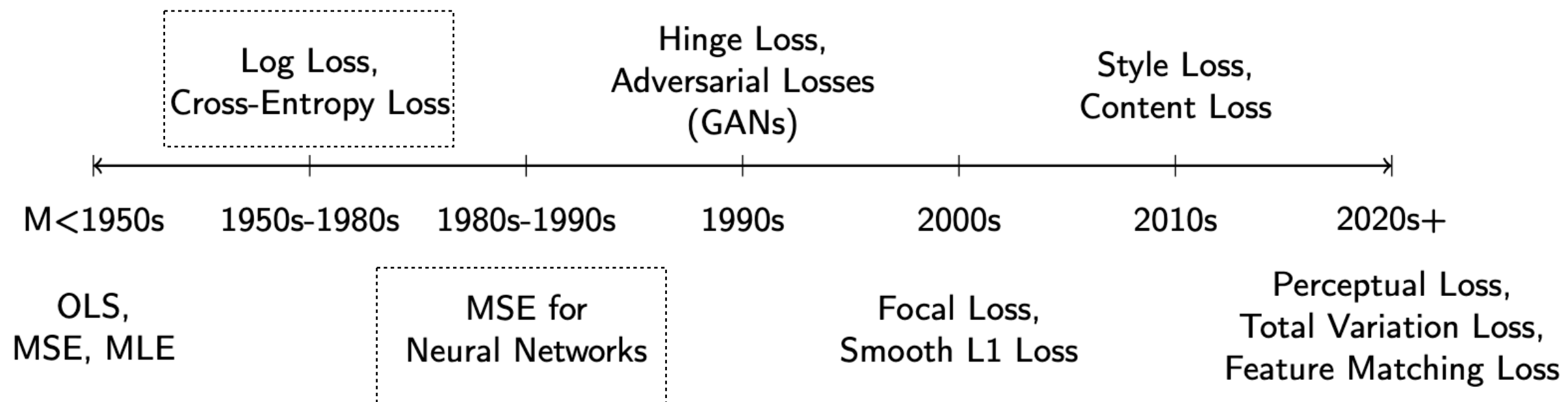
Problem	Model	Loss	Activation	PyTorch
Regression	Gaussian	$\mathcal{L}_{\mathcal{D}}^{SSE}$	Identity	<code>nn.MSELoss</code>
Binary Classification	Bernoulli	$\mathcal{L}_{\mathcal{D}}^{BCE}$	Sigmoid	<code>nn.BCEWithLogitsLoss</code>
Multi-class Classification	Categorical	$\mathcal{L}_{\mathcal{D}}^{CE}$	Softmax	<code>nn.CrossEntropyLoss</code>

Engineering the Loss

Beyond the cookbook

Weights Learn Their Purpose

The **weights are initially meaningless**. Training with a specific loss function induces their meaning - they **learn to represent what the loss requires**



Timeline of evolution of loss functions, see [Elharrouss+, arXiv \(2025\)](#) for a comprehensive review

Loss as Proxy

The loss is often not the actual metric we care about, but a proxy that's **easier to optimise** — e.g., differentiable

Example: cross-entropy (smooth, differentiable) vs. accuracy (discrete, non-differentiable)

Engineer the Objective

The loss can be crafted for a **specific task** or terms may be added to **enforce desired constraints**

Weight decay for regularisation (Lecture 5), fairness constraints, physical laws, ...

$$\mathcal{L}_{\mathcal{D}} = \mathcal{L}_{\mathcal{D}}^{(0)} + \mathcal{L}_{\mathcal{D}}^{physics} + \dots$$

Lecture Report

3 : Losses



Key concepts:

- ✓ **Loss functions**
General strategy, engineering
- ✓ **Regression**
Sum-of-squares error + ID
- ✓ **Binary classification**
Binary cross-entropy + sigmoid
- ✓ **Multi-class classification**
Cross entropy + softmax

Bibliography:

- Bishop+ (2023), Ch. 6

Exercises:

- Bishop+ (2023), Ex. 6.14, 6.15

Next week:

Gradient Descent